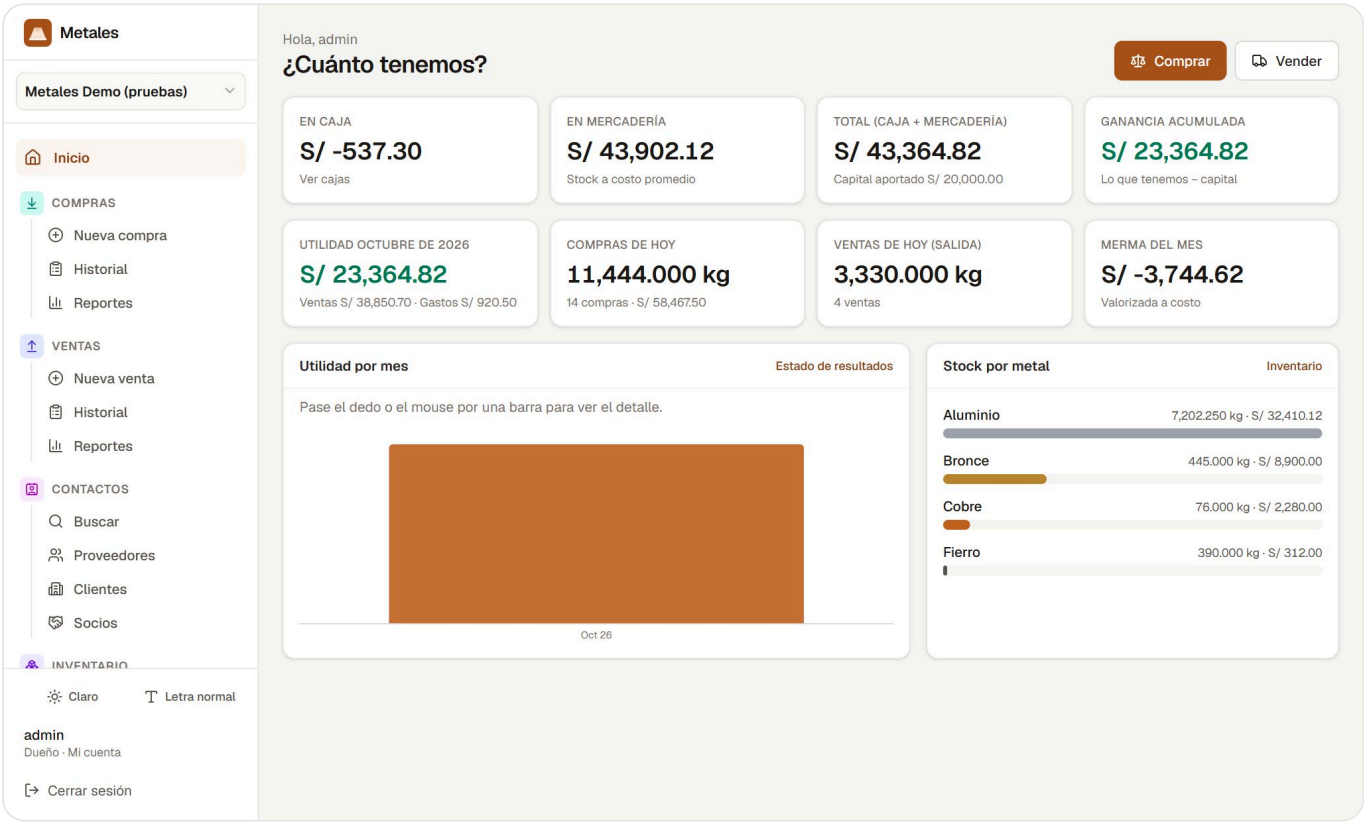
Sistema Metales

Documentación de estudio: qué es el sistema, cómo funciona por dentro y qué se construyó, paso a paso.



Proyecto: **Proyecto_Otiniano** · Negocio de compra-venta de metales (chatarra), Perú
Tecnologías: Next.js 15 · React 19 · TypeScript · Tailwind CSS 4 · Supabase (Postgres) · pnpm
Versión del documento: 16 de octubre de 2026 · Las capturas usan la empresa de **datos de prueba**.

Contenido

1. El negocio y el problema que resolvemos
2. Qué es el sistema (y qué no es)
3. Recorrido por las pantallas
4. Roles: quién puede hacer qué
5. Conceptos del negocio explicados con números
6. Arquitectura: las piezas y para qué sirve cada una
7. El viaje de una compra: del botón a la base de datos
8. La base de datos por dentro
9. Seguridad
10. Estructura de carpetas del proyecto
11. Pruebas automáticas
12. Qué se hizo, en orden (bitácora del proyecto)
13. Cómo se trabaja en el día a día
14. Pendientes y decisiones abiertas
15. Ruta de estudio y glosario técnico

Cómo leer este documento

Los capítulos 1 a 5 explican el **negocio** y el uso del sistema: sirven para hablar con el cliente. Los capítulos 6 a 11 explican la **parte técnica**: sirven para entender el código y poder modificarlo. Los capítulos 12 a 15 cuentan **qué se hizo**, qué falta y qué estudiar para dominar el proyecto.

1. El negocio y el problema

El cliente (Otiniano) tiene con un primo, **en sociedad**, un negocio de compra y venta de metales: cobre, bronce, aluminio, fierro. Así funciona hoy:

- Al local llega **gente de a pie** a venderles metal: desde un kilito de cobre hasta una tonelada. Esa gente **no emite factura**.
- El negocio acumula el metal y luego lo **vende a un comprador mayor** (fundiciones, recicladoras). Para trasladarlo hacen una **guía**.
- Siempre hay **merma**: lo que se compró no pesa exactamente lo mismo cuando se vende.
- Como es una sociedad, a fin de mes necesitan saber **si ganaron o perdieron**: «iniciamos con 20 mil y ahora tenemos 21, 22 o 23 mil; hay tanto en plata y el resto en producto».
- Quieren **registrar a quienes les venden** para, más adelante, premiarlos (por ejemplo, un sorteo navideño).

> **El objetivo en una frase**
>
> Que en cualquier momento el dueño vea **cuánto tiene en caja**, **cuánto en mercadería** (kilos × costo) y **cuánto ganó**, por metal y por mes, con un registro de compras rapidísimo de hacer en el mostrador desde el celular.

La decisión de diseño más importante: multiempresa

El cliente preguntó si existía algo tipo SaaS («uno se registra y ya hace sus movimientos»). Se decidió construirlo **a medida para él**, pero **multiempresa desde el primer día**: cada dato lleva el identificador de su empresa (`empresa_id`). Así, el día de mañana se le puede ofrecer a otros negocios sin rehacer nada. Por eso existen hoy dos empresas en la base: la real («Metales Otiniano», vacía) y una de pruebas («Metales Demo»), y ninguna ve los datos de la otra.

2. Qué es el sistema (y qué no es)

ES	DETALLE
Una aplicación web	Se abre en el navegador. No hay que instalar nada en la computadora.
Instalable en el celular (PWA)	Desde el navegador del celular se puede «Agregar a pantalla de inicio» y queda como una app, con su ícono.
Privada	No tiene página web pública ni catálogo. Solo entra quien tiene usuario y contraseña creados por el dueño.
Pensada para el celular primero	Botones grandes, la compra se hace con el pulgar. En computadora se aprovecha para ver reportes.
Multiempresa	Un mismo usuario puede pertenecer a varias empresas y cambiar entre ellas.

NO ES (TODAVÍA)	POR QUÉ
Facturación electrónica SUNAT	Quedó para la fase 4. Hoy se guarda el número y la foto de la guía.
Ventas a crédito, en dólares o con varios locales	Son preguntas abiertas al cliente (**[CONFIRMAR]**). No se inventan: se espera su respuesta.
Sorteos para proveedores	Fase 4. Ya existe el ranking de proveedores que servirá de base.

Cómo se organiza la pantalla

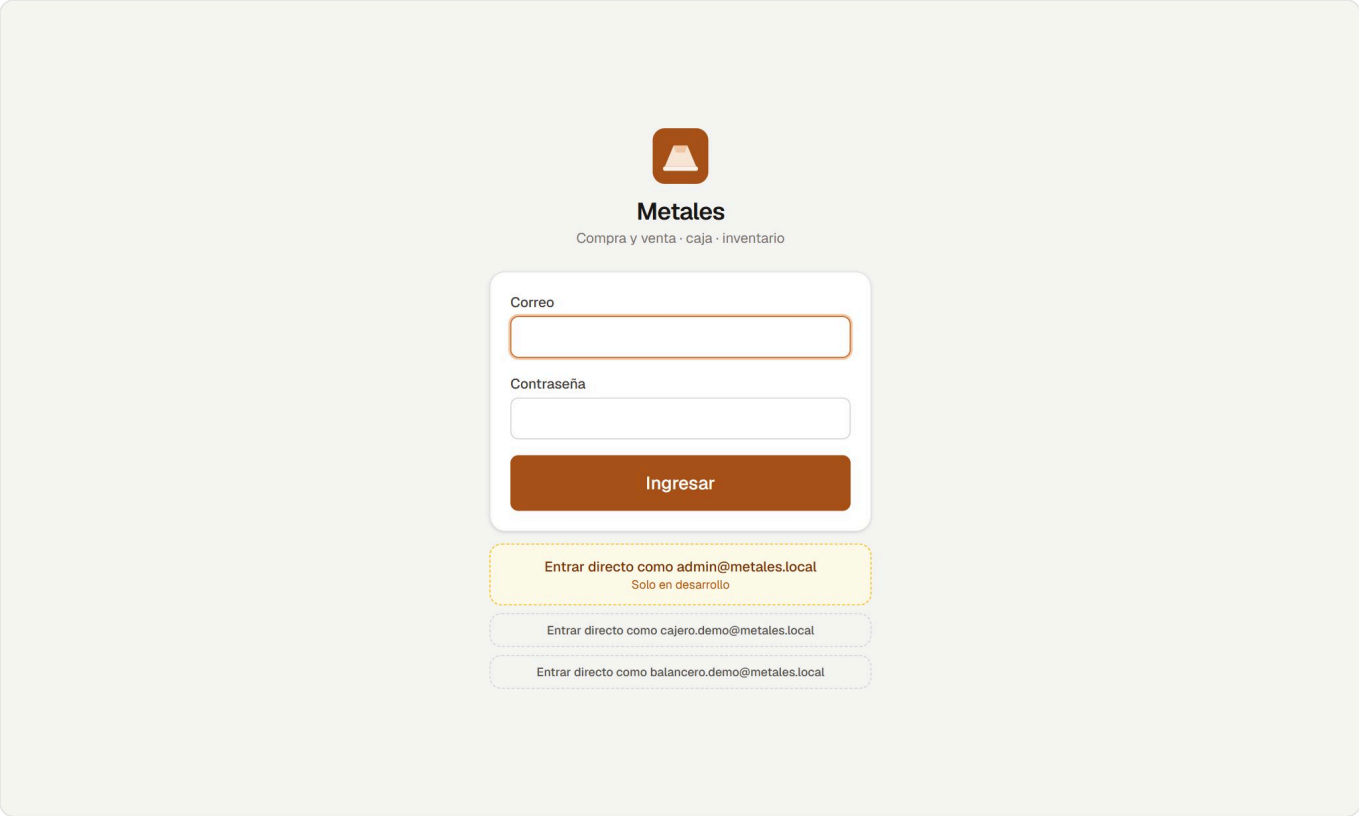
Todo está agrupado en **módulos**, cada uno con su color para ubicarse rápido. Dentro de cada módulo hay **pestañas** arriba (por ejemplo: Nueva compra · Historial · Reportes). Los colores verde, rojo y ámbar están reservados para **estados** (registrada, anulada, pendiente), por eso ningún módulo los usa.

MÓDULO	PESTAÑAS
Inicio	Tablero del dueño; para cajero y balancero, accesos grandes a lo que hacen cada día.
Compras	Nueva compra · Historial · Reportes
Ventas	Nueva venta · Historial · Reportes
Contactos	Buscar · Proveedores · Clientes · Socios
Inventario	Stock · Precios del día · Reportes
Caja y finanzas	Caja · Capital y cierres · Reportes
Gestión	Todos los reportes · Configuración

La lista de módulos vive en un solo archivo: `src/components/shell/modulos.ts`. El menú lateral, las pestañas y la barra inferior del celular se generan desde ahí.

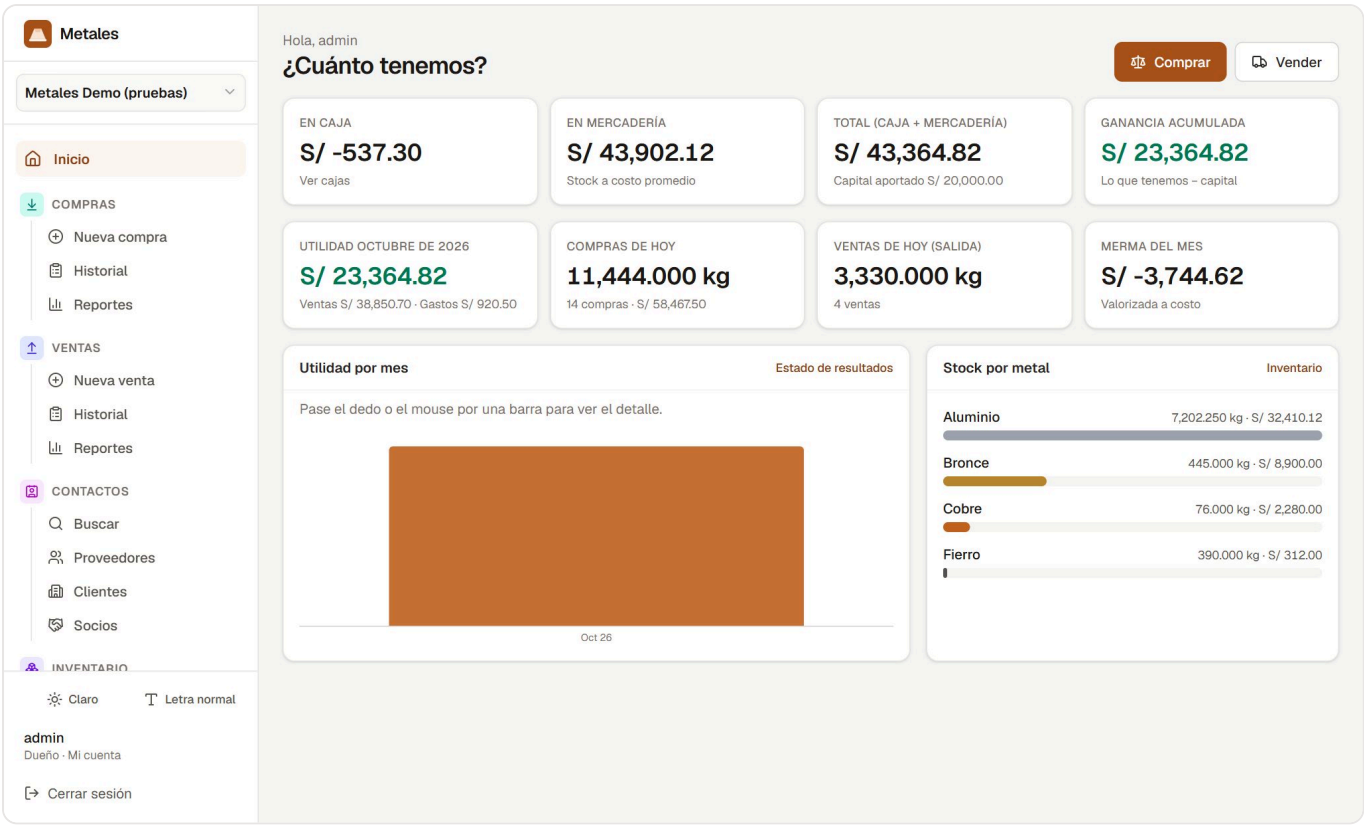
3. Recorrido por las pantallas

Entrada al sistema



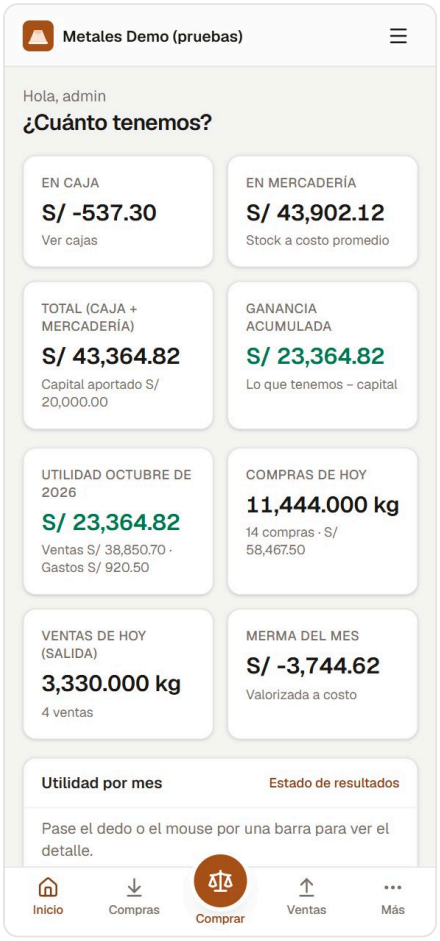
Login. En desarrollo aparece además un acceso directo de prueba; en producción ese acceso no existe (devuelve 404).

Inicio del dueño: «¿Cuánto tenemos?»

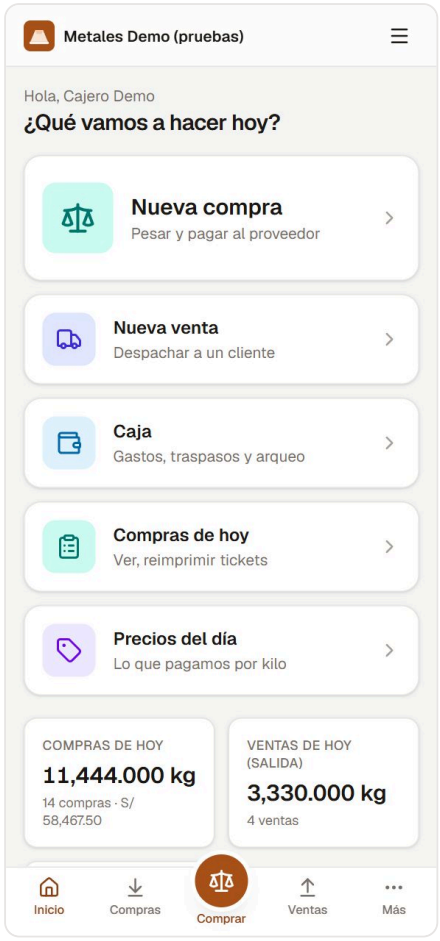


Caja + mercadería = total; total − capital aportado = ganancia acumulada. Abajo, la utilidad por mes y el stock por metal.

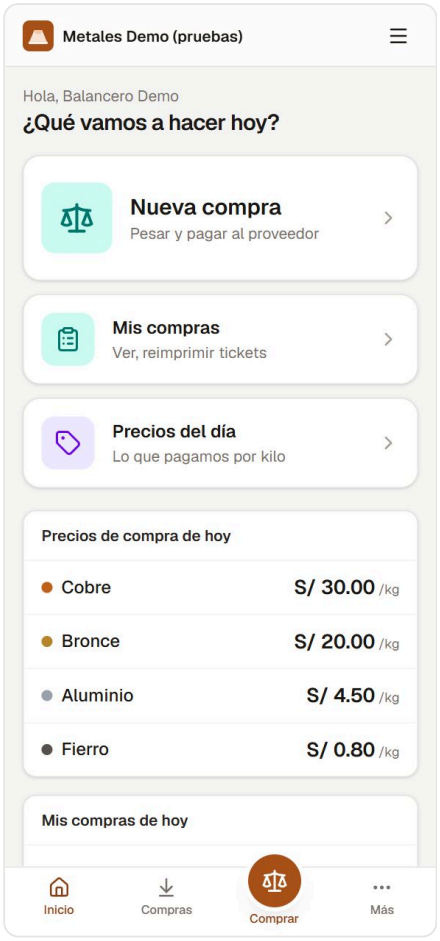
En el celular: lo que ve cada rol al entrar



Dueño



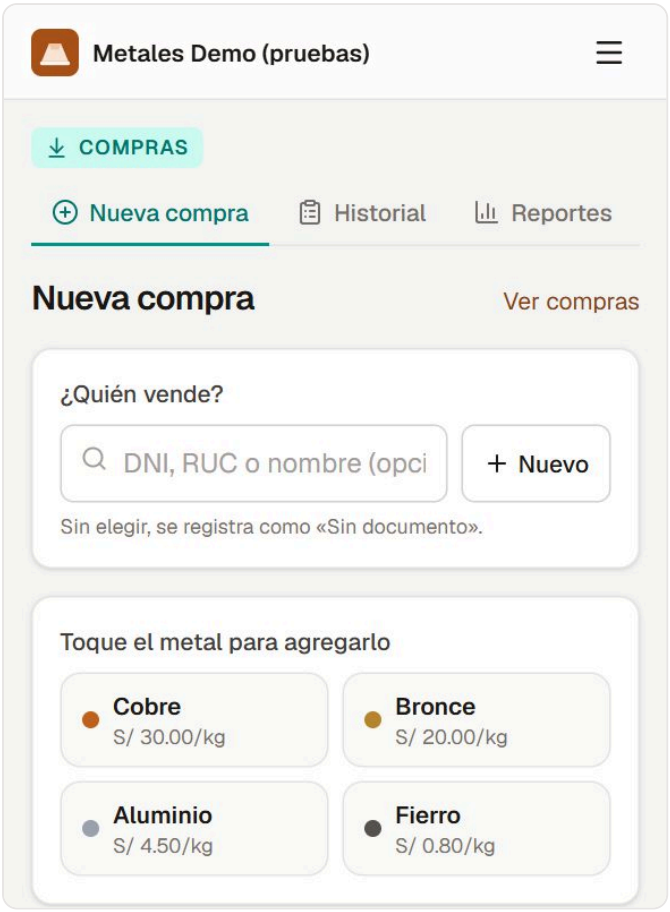
Cajero: accesos grandes



Balancero: solo compras

Abajo hay una barra fija con **Inicio · Compras · Comprar (botón central grande) · Ventas · Más**. El botón central es la acción más usada del negocio.

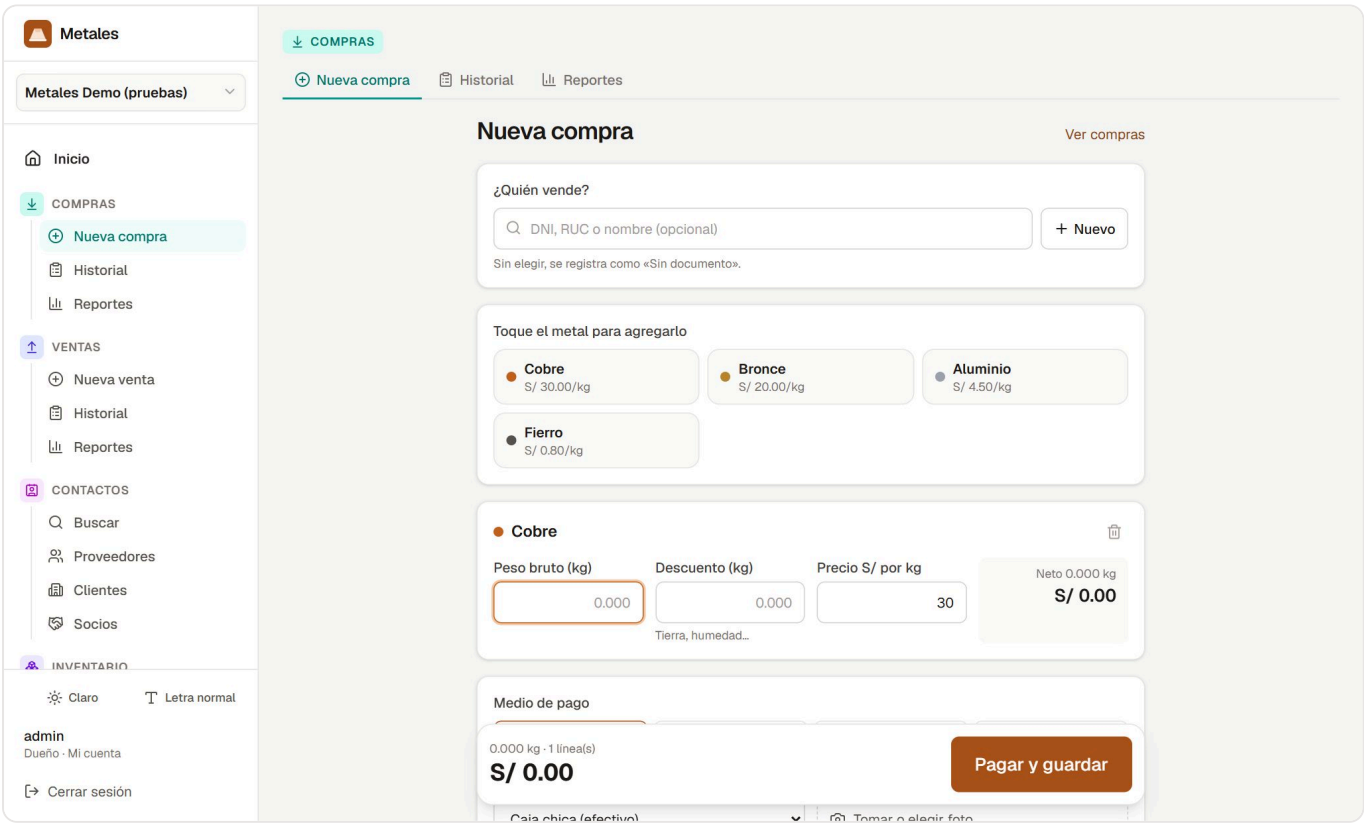
Compras Registrar lo que nos venden



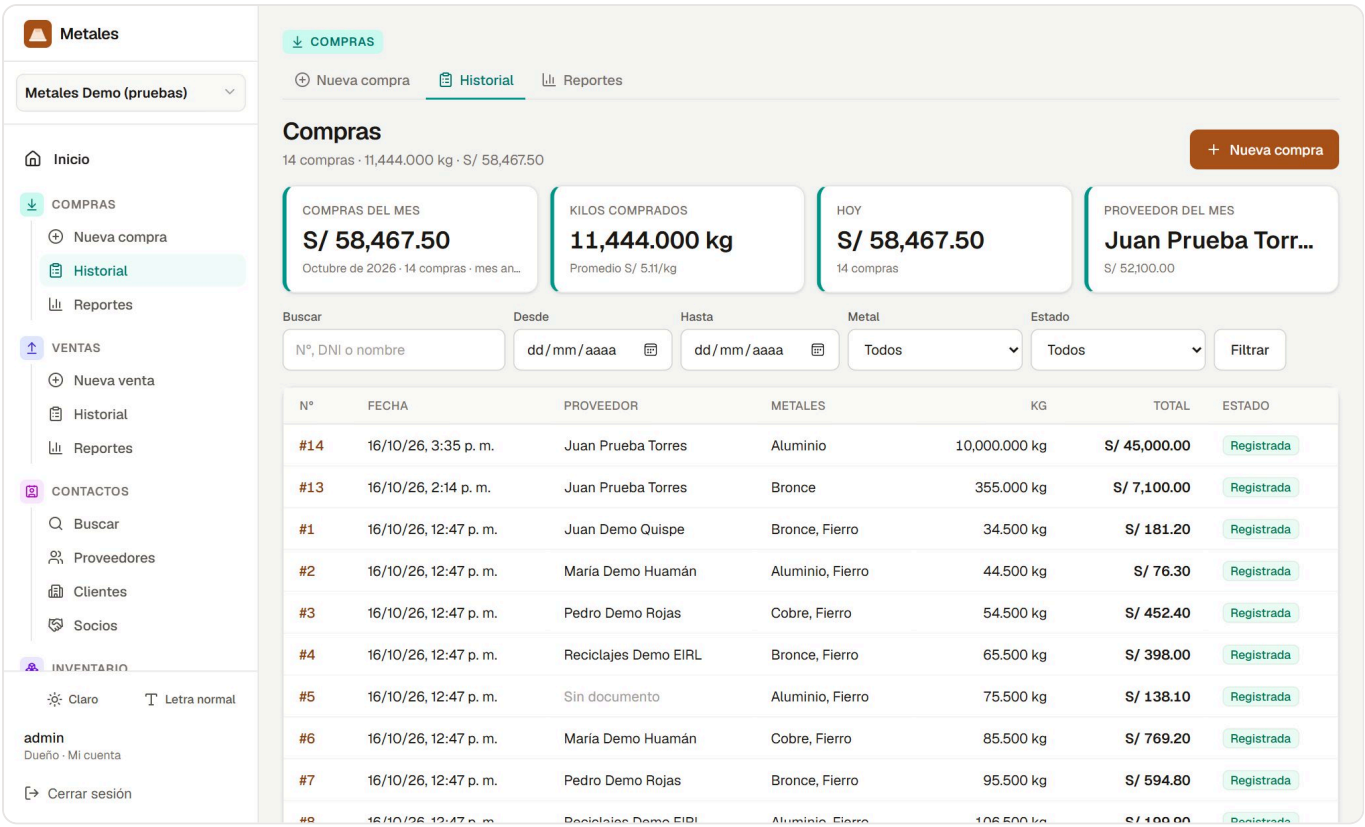
Nueva compra en el celular: se toca el metal y se agrega.

Pasos de una compra:

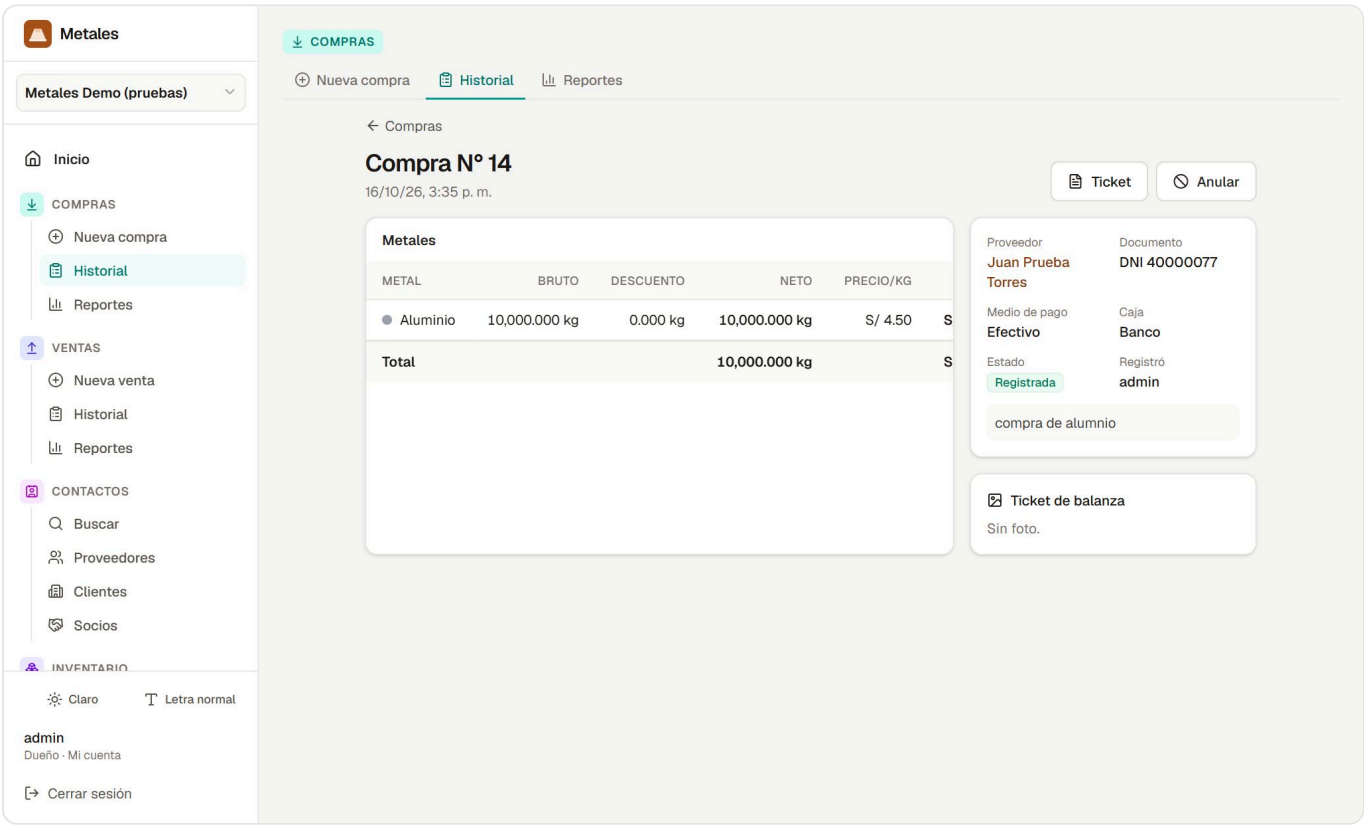
1. **¿Quién vende?** Se busca por DNI, RUC o nombre. Si no existe, se crea ahí mismo con «+ Nuevo». Si se deja vacío, queda como «Sin documento» (compras chicas).
2. **Metal:** se toca la tarjeta (Cobre, Bronce…). El precio del día ya viene puesto.
3. **Pesos:** peso bruto − descuento (tierra, humedad) = **peso neto**.
4. **Total en vivo** mientras se escribe.
5. **Medio de pago** y caja (efectivo, Yape…), foto opcional del ticket de balanza.
6. **Pagar y guardar** → sale el ticket de 80 mm en PDF.



La misma pantalla en computadora, con una línea de cobre agregada.



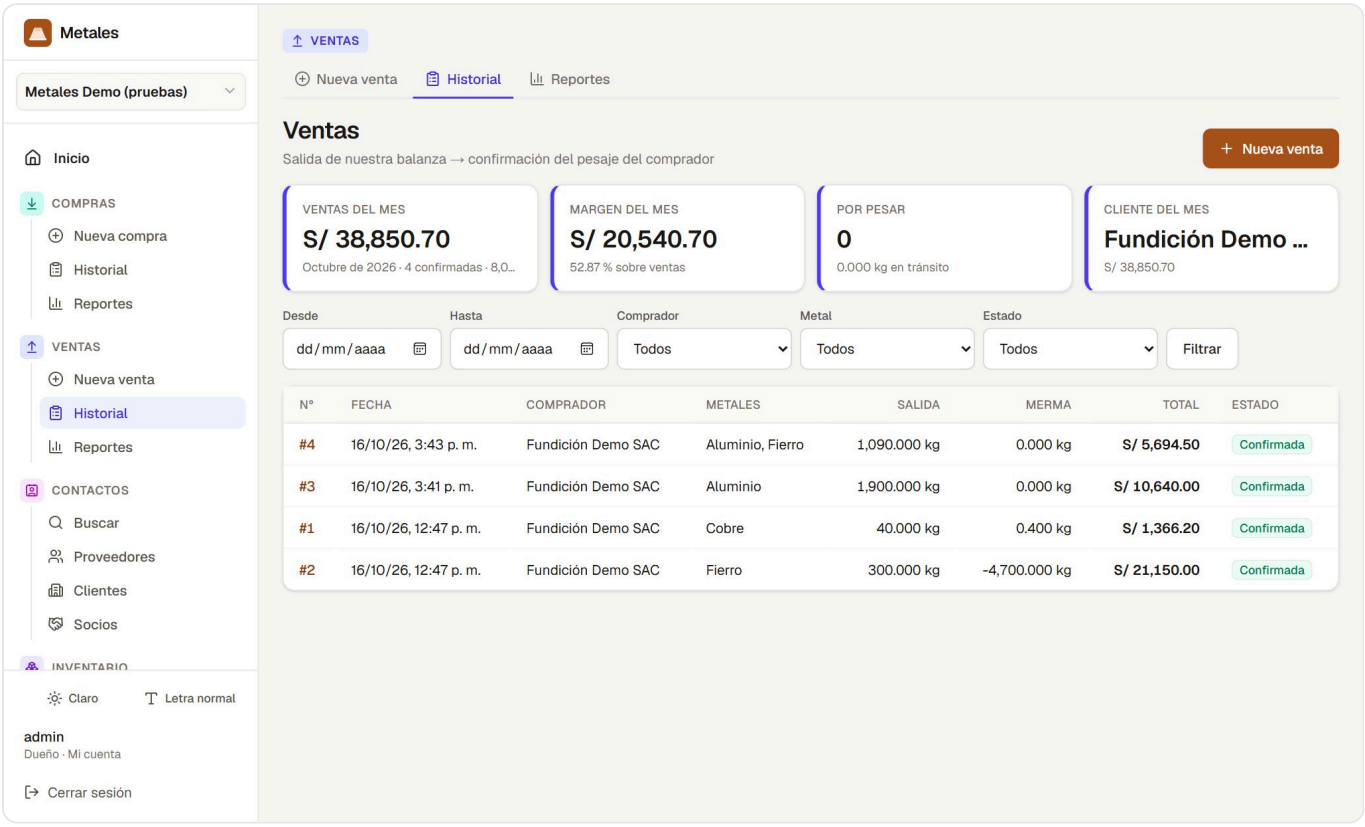
Historial de compras: resumen del mes arriba (cuánto, cuántos kg, precio promedio, proveedor top) y filtros por fecha, proveedor, metal y estado.



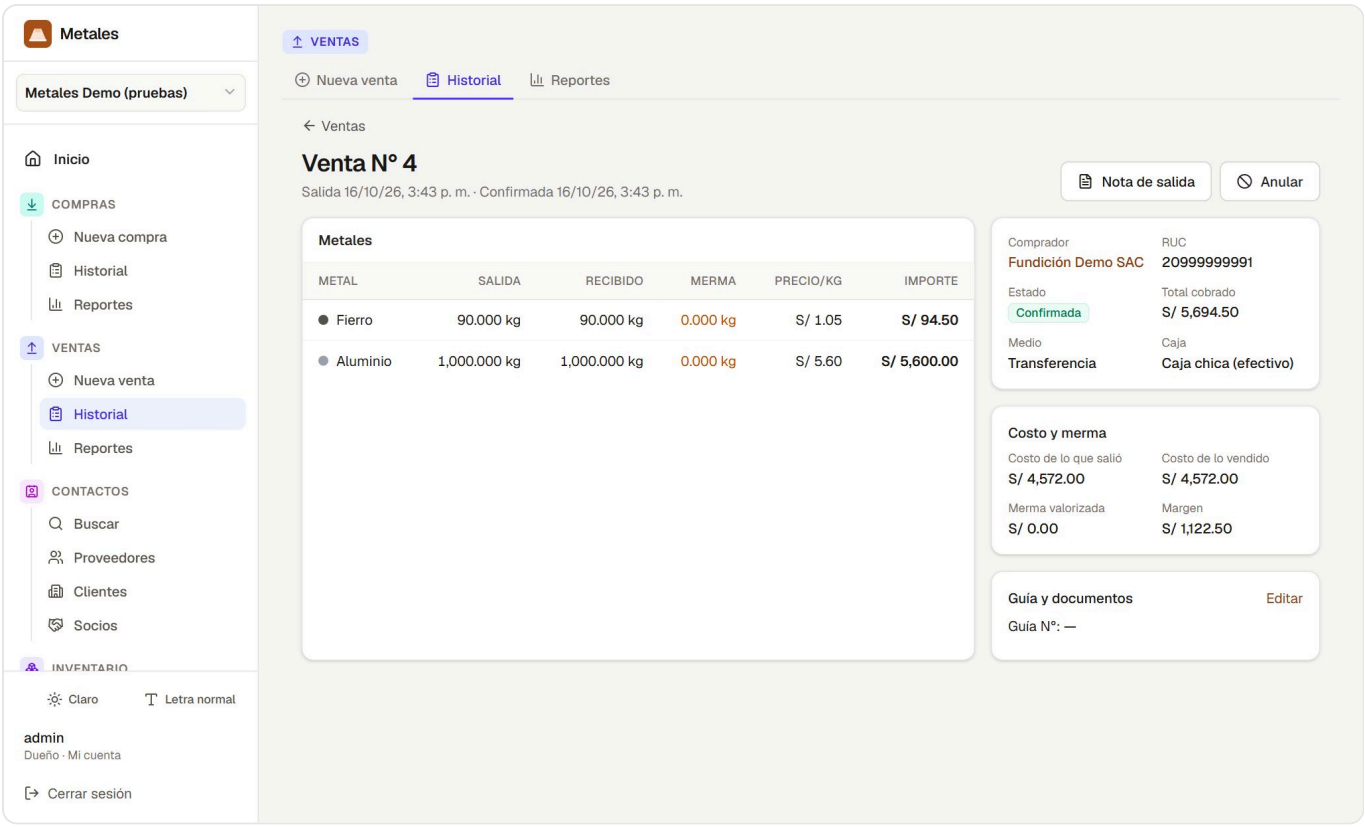
Detalle de una compra: líneas, totales, ticket PDF y botón para anular (solo el dueño, con motivo obligatorio).

Ventas Despachar al comprador mayor

Una venta tiene **dos momentos**: (1) se registra con el **peso de salida** (nuestra balanza) y queda *pendiente*; el stock ya sale. (2) Cuando el comprador pesa en su balanza, se **confirma** con el **peso recibido** y el precio. La diferencia es la **merma de venta** y se registra sola. Recién ahí entra el dinero a caja.



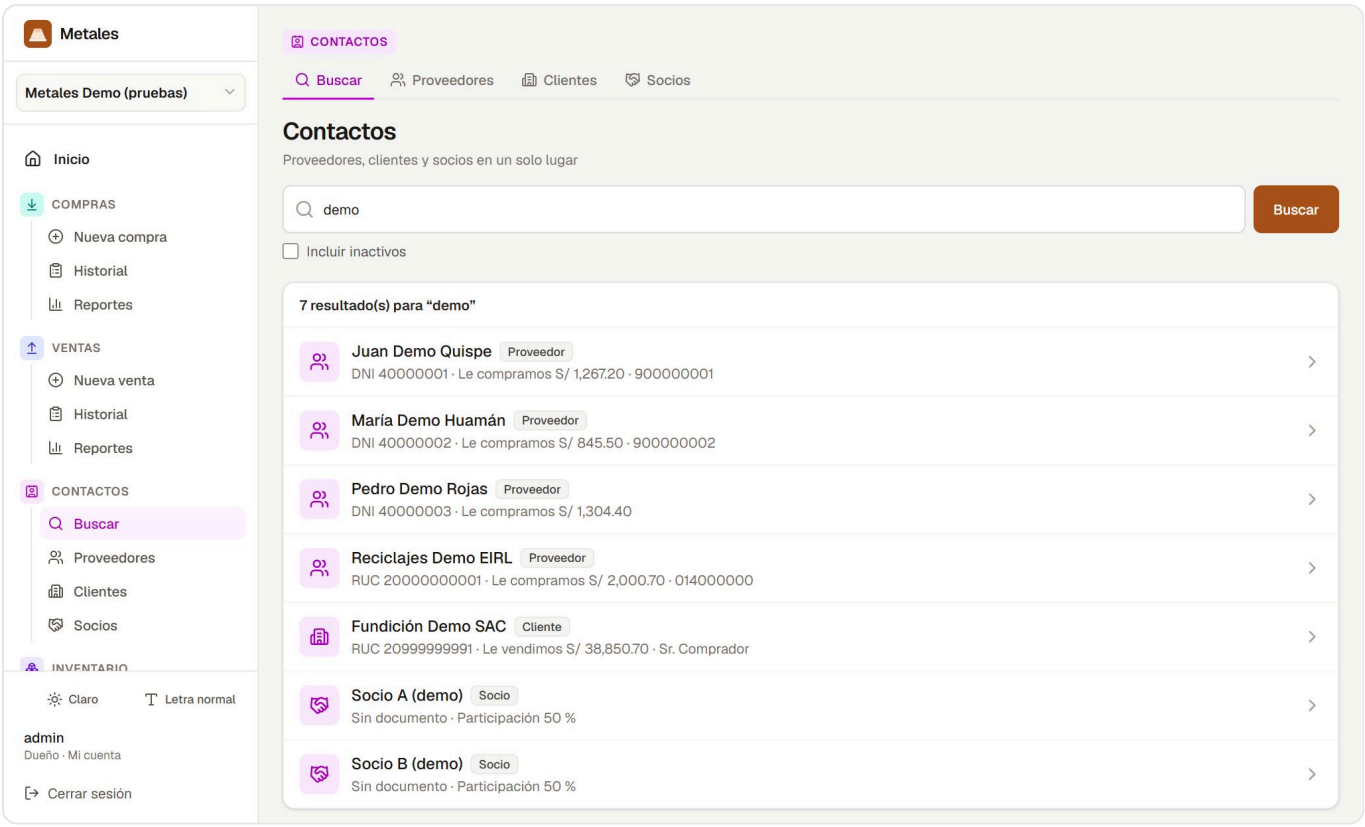
Historial de ventas con resumen del mes: importe, margen, merma y ventas pendientes de confirmar.



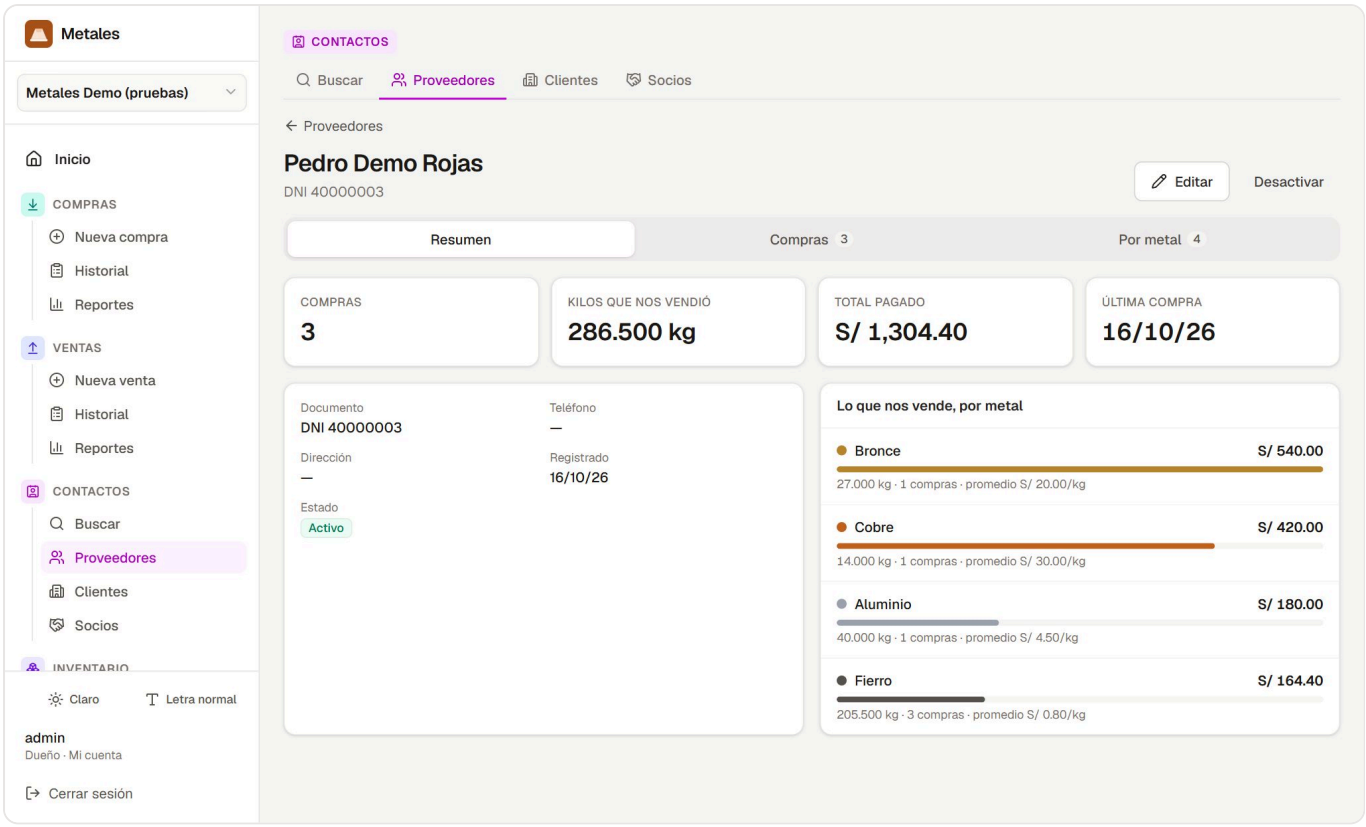
Detalle de una venta: kg de salida, recibidos, merma, precio y margen por línea.

Contactos Proveedores, clientes y socios

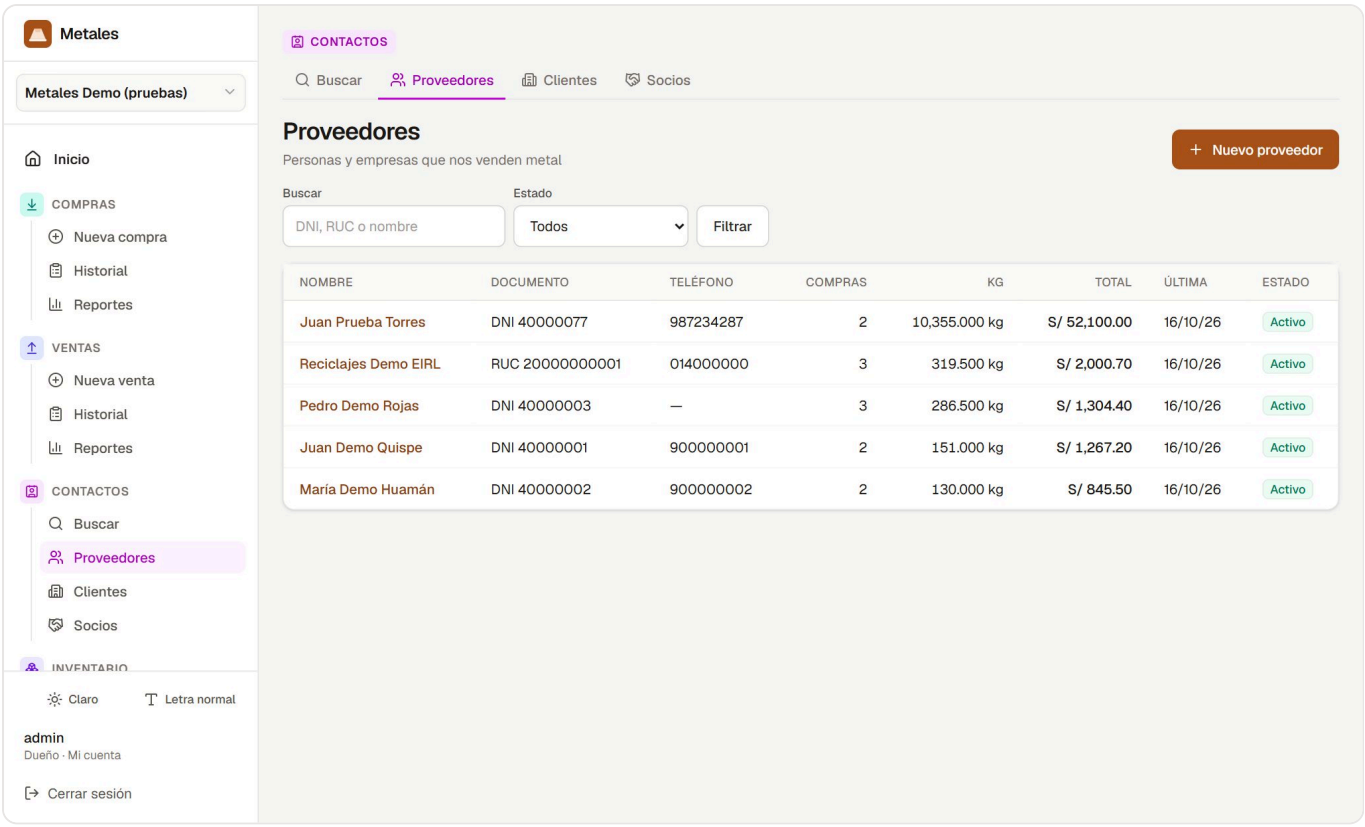
Un solo buscador: se escribe un **DNI, RUC o nombre** y aparece donde esté. Si son solo números busca por documento; si hay letras, por nombre. Sin escribir nada muestra cuántos activos hay de cada tipo.



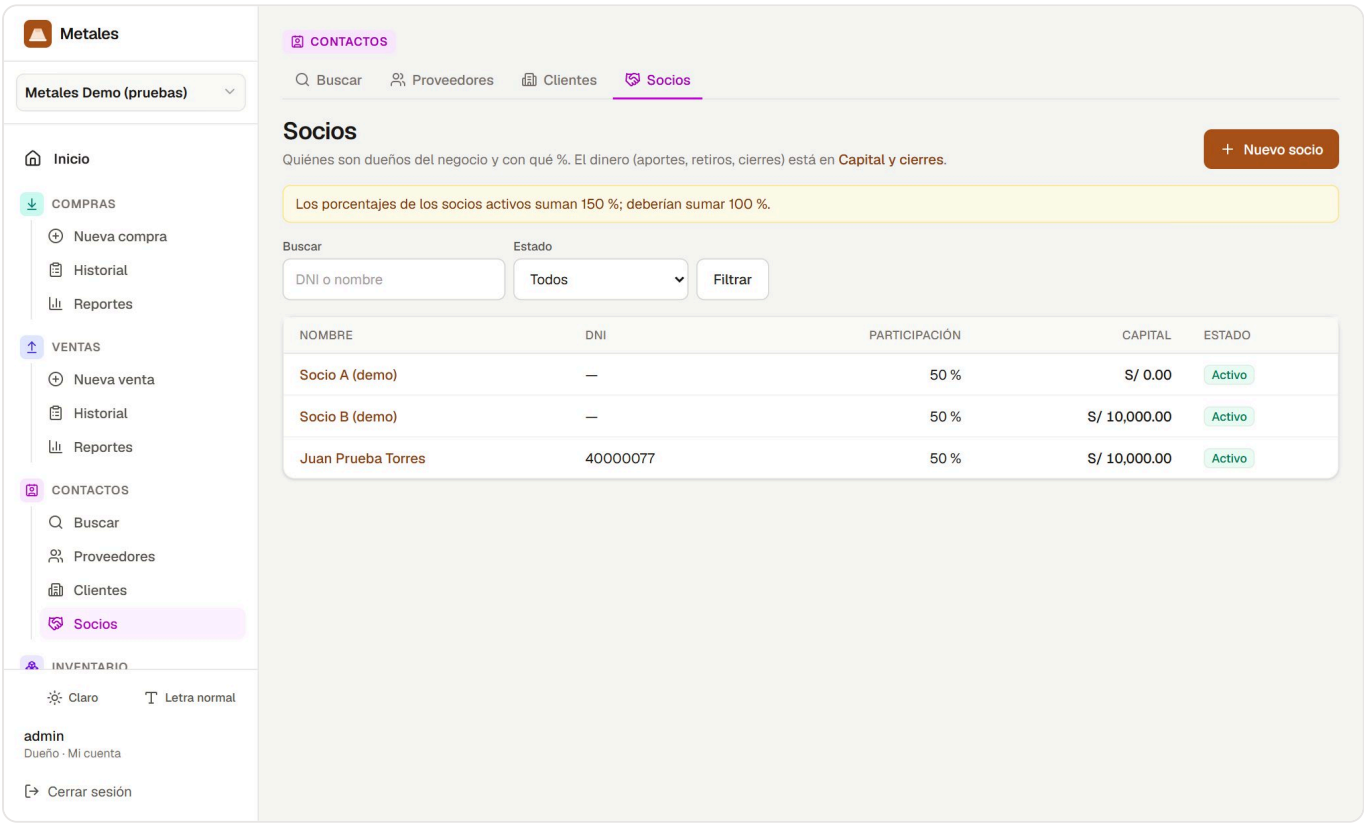
Búsqueda «demo»: resultados de proveedores, clientes y socios con su documento y cuánto se les compró o vendió.



Ficha de un proveedor, pestaña «Por metal»: cuántos kg y soles nos ha vendido de cada metal.

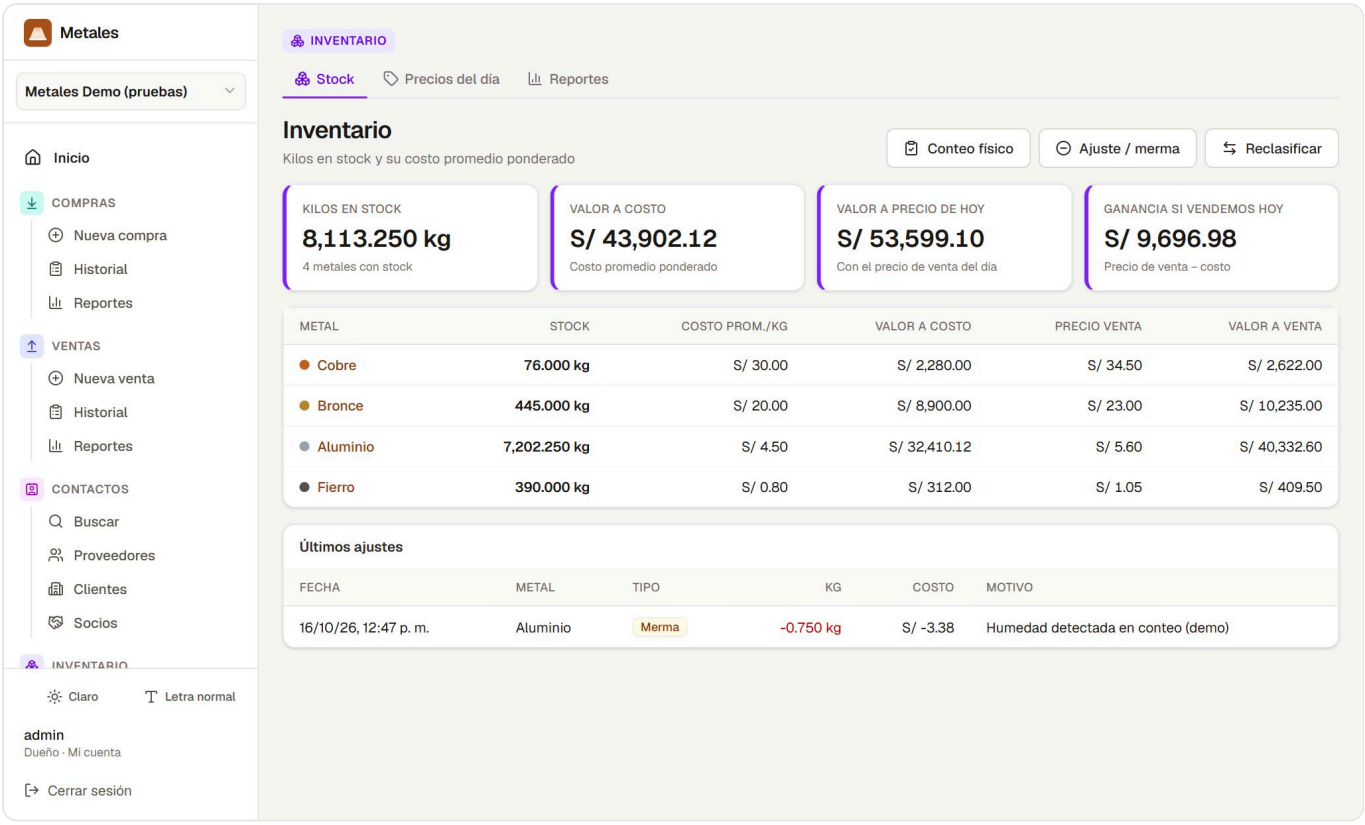


Proveedores: filtro de activos/inactivos y buscador.

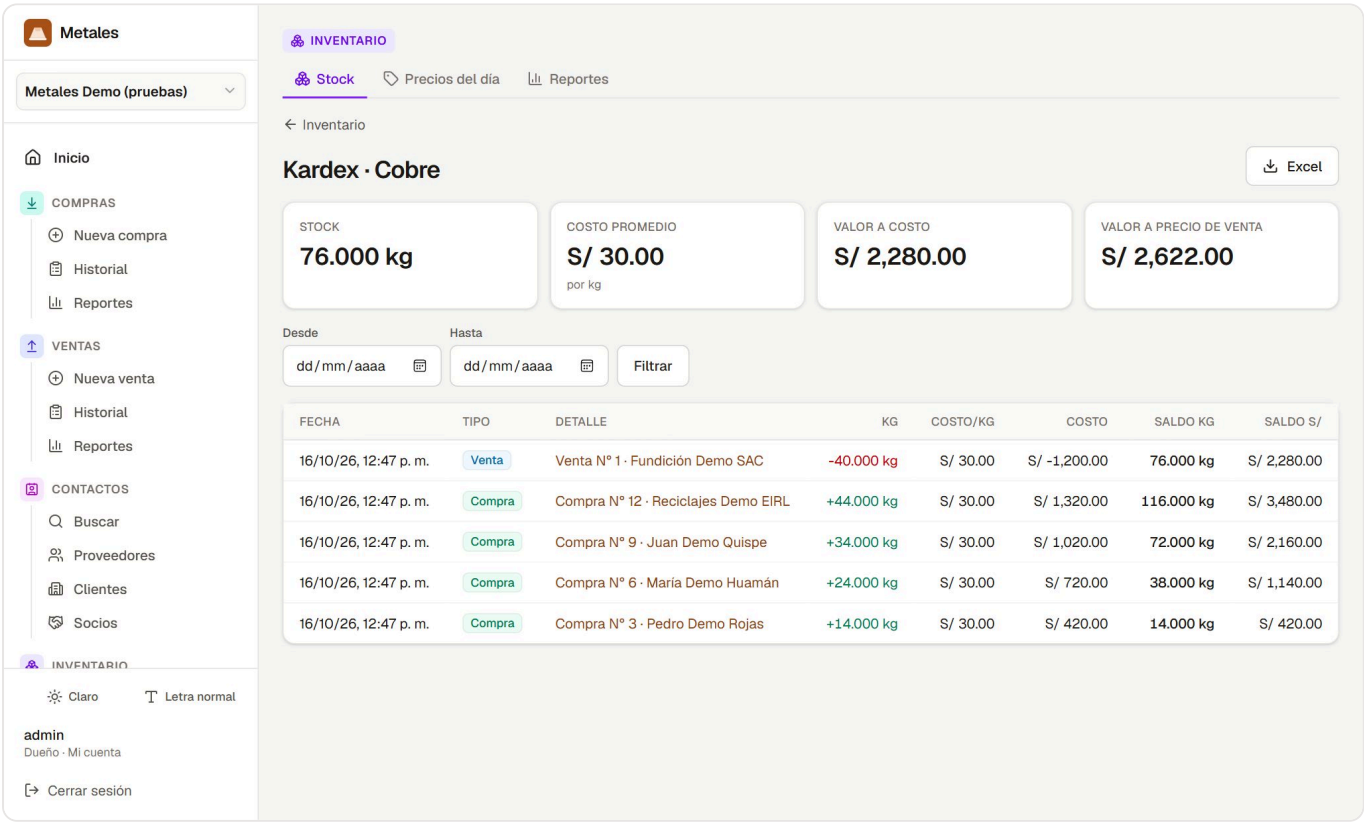


Socios: quiénes son dueños y con qué %. Avisa si los porcentajes de los activos no suman 100 %.

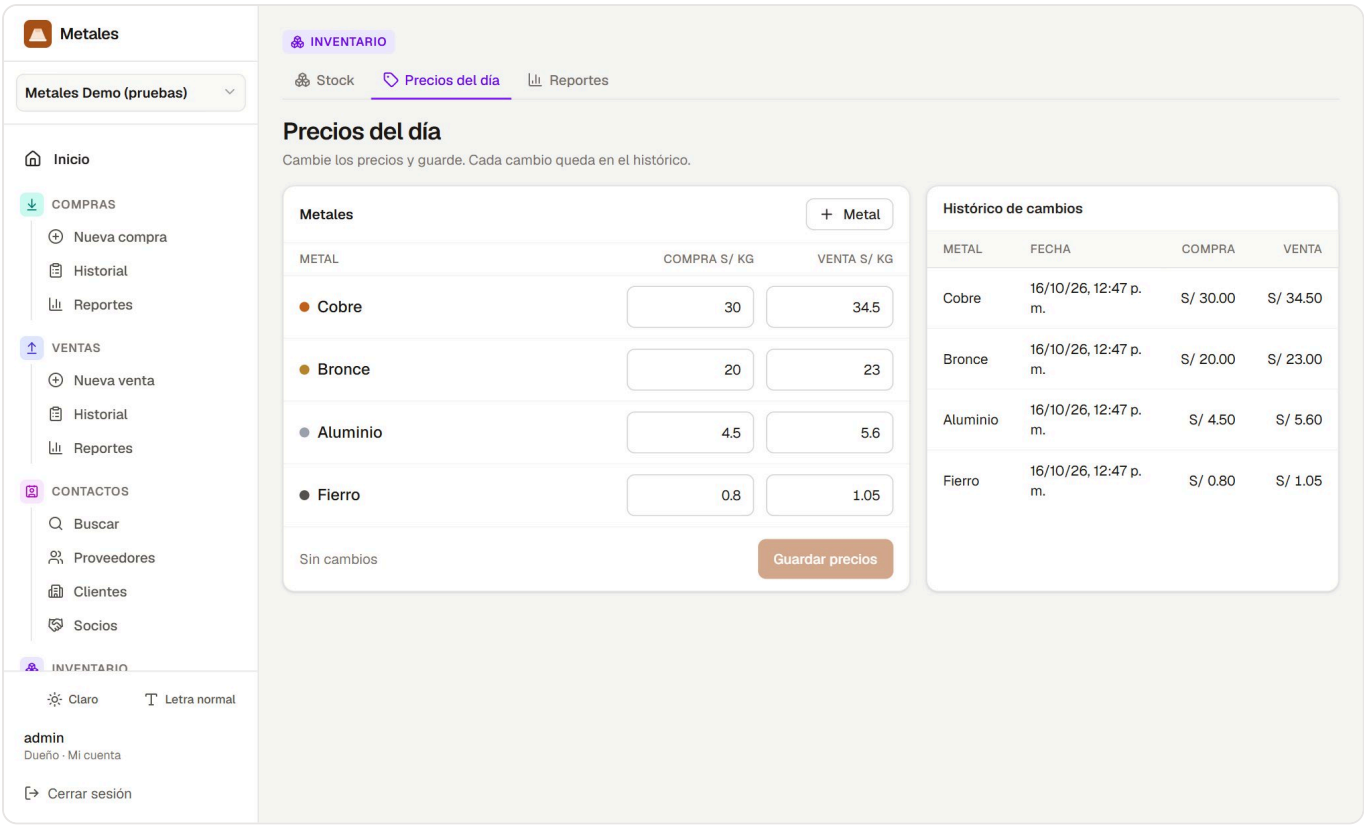
Inventario Cuánto metal tenemos y cuánto vale



Stock por metal: kg, costo promedio, valor a costo y a precio de venta, y ganancia potencial.

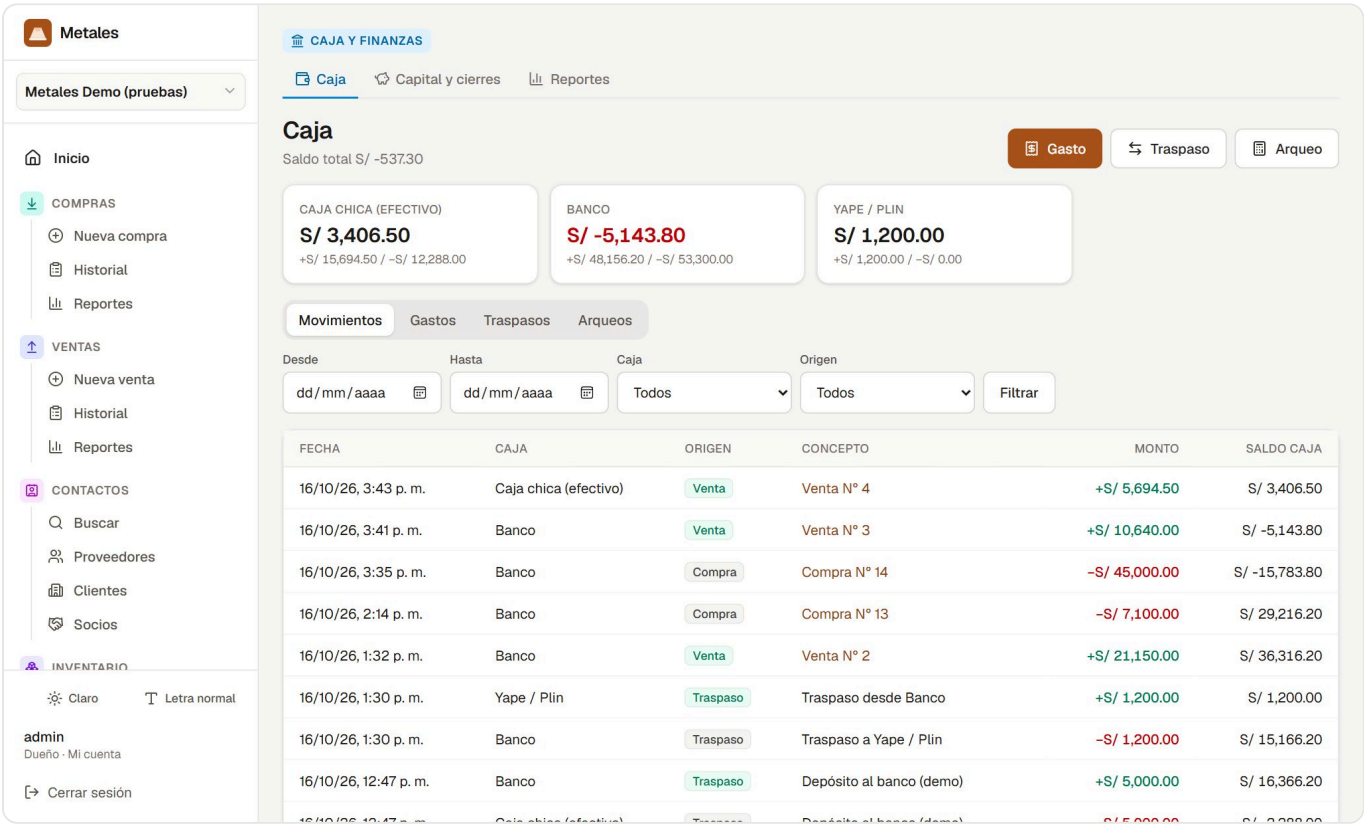


Kardex del cobre: cada entrada (compra) y salida (venta, ajuste) con su costo y el saldo resultante. Es el «libro» del metal.

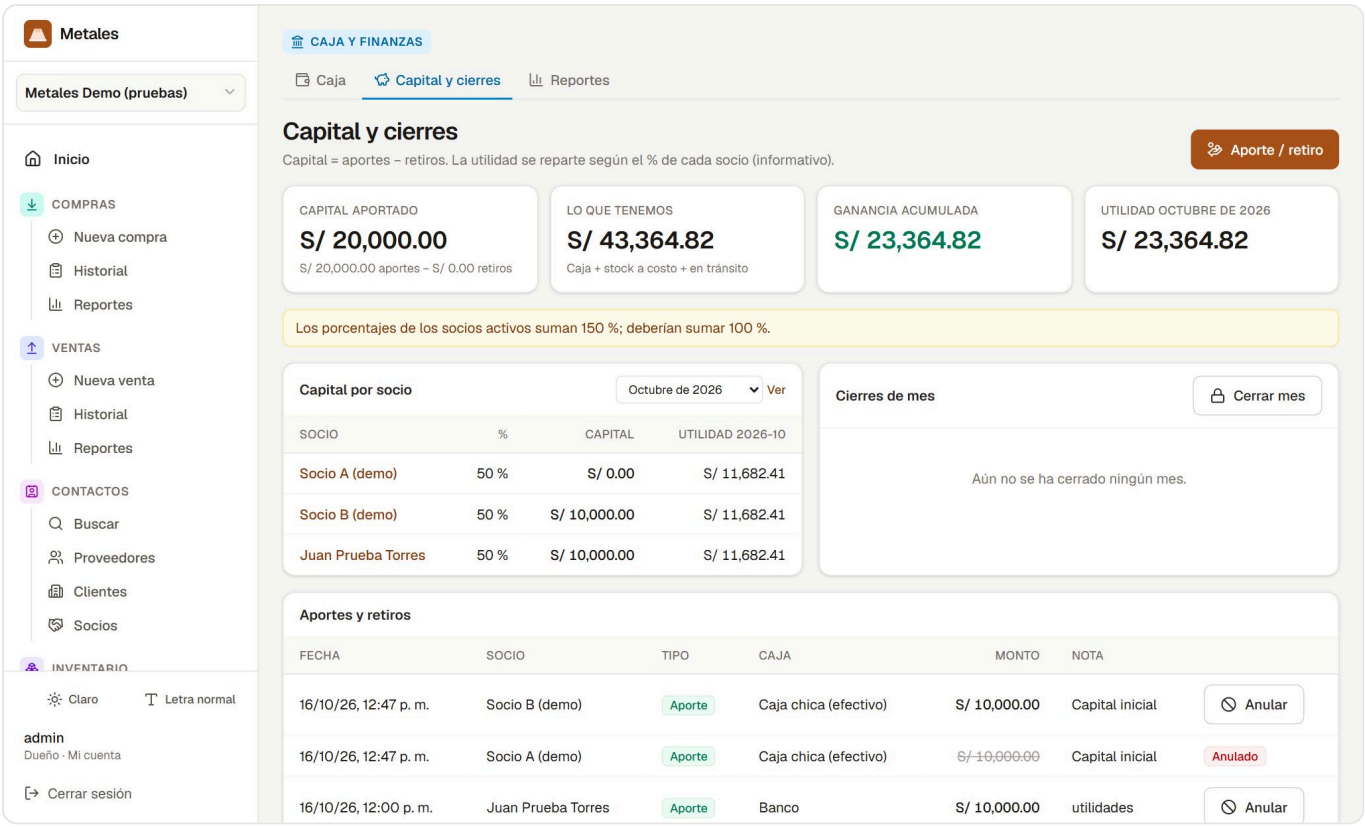


Precios del día: lo que pagamos y cobramos por kilo. Cambiarlos queda en el histórico. Solo el dueño los edita.

Caja y finanzas El dinero

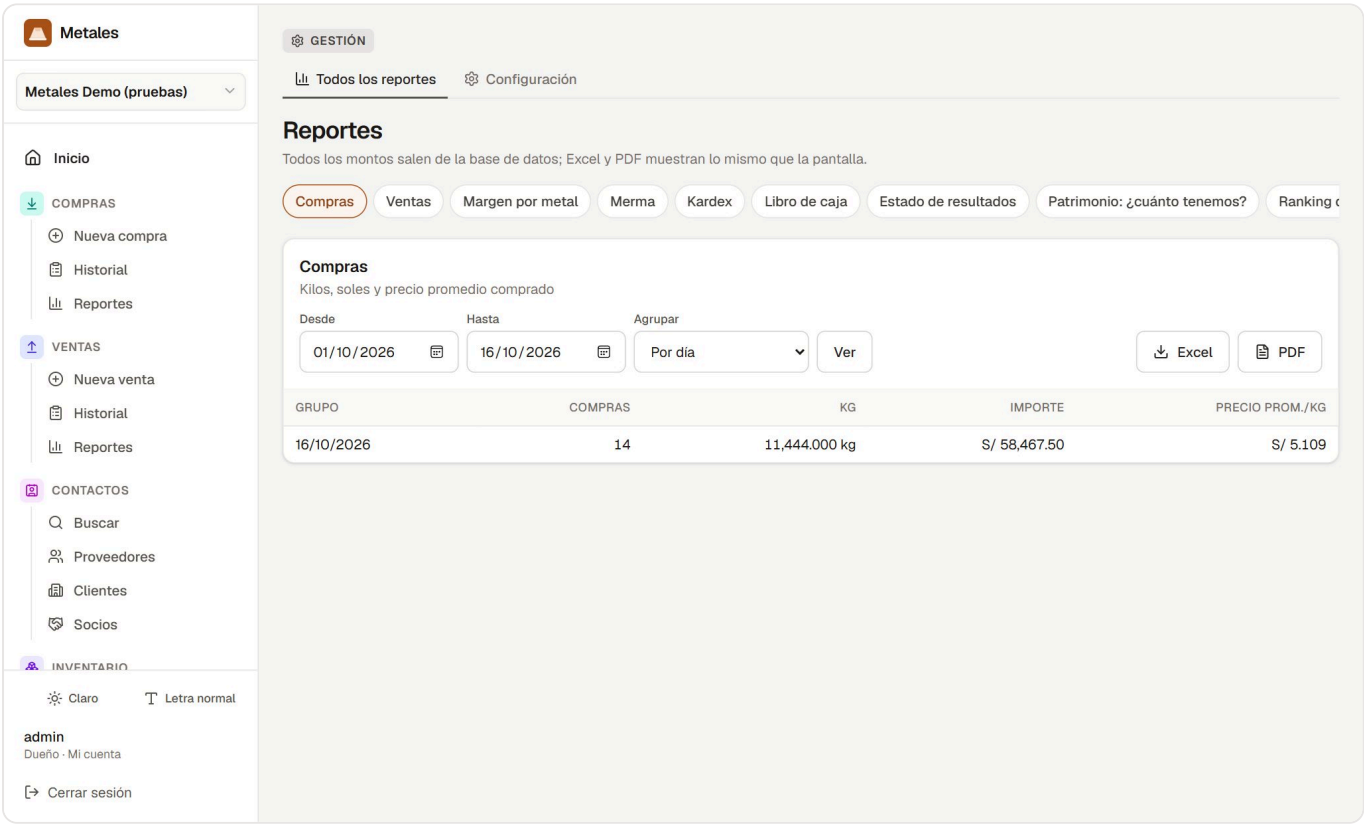


Caja: saldo por caja, movimientos, gastos, traspasos entre cajas y arqueo.

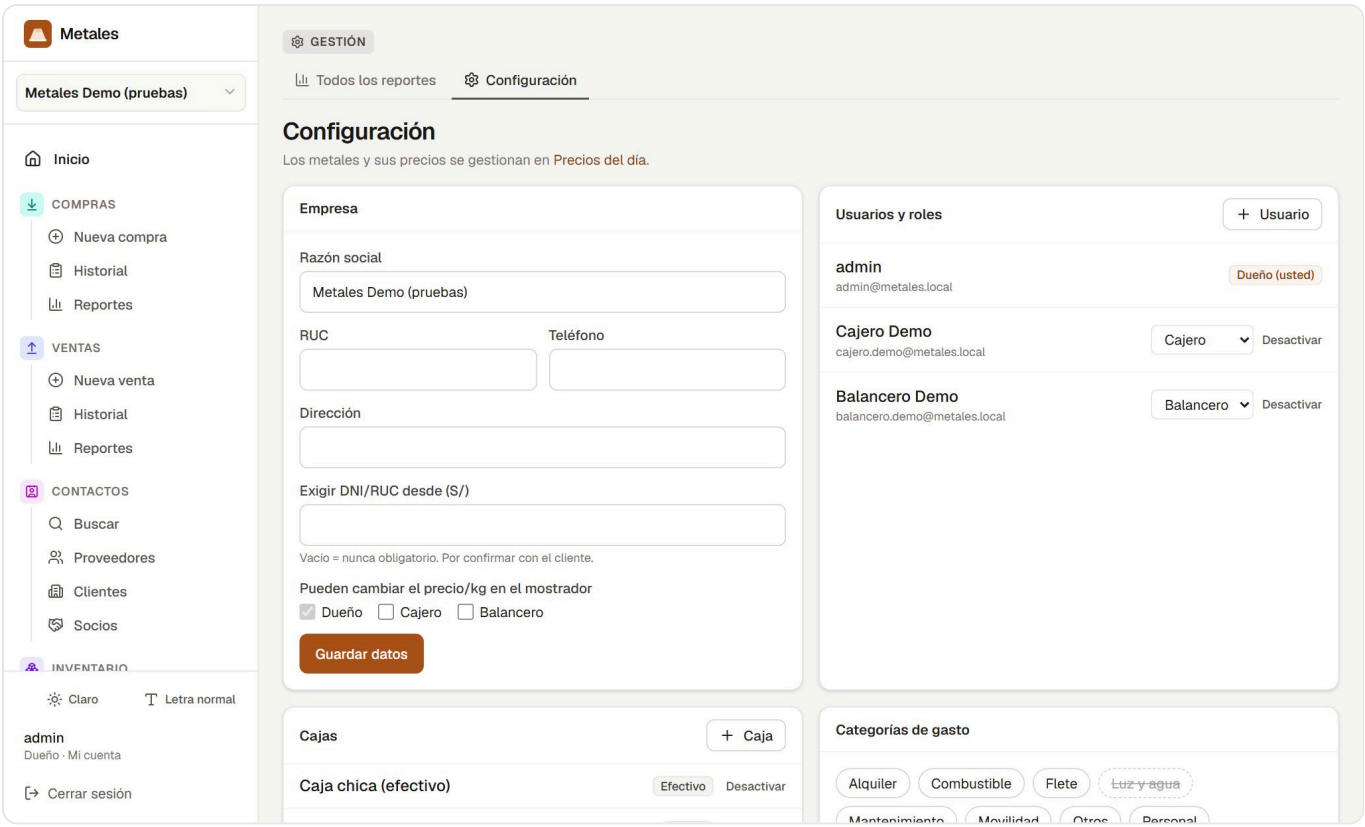


Capital y cierres: capital por socio (aportes − retiros), cierres de mes y movimientos de los socios.

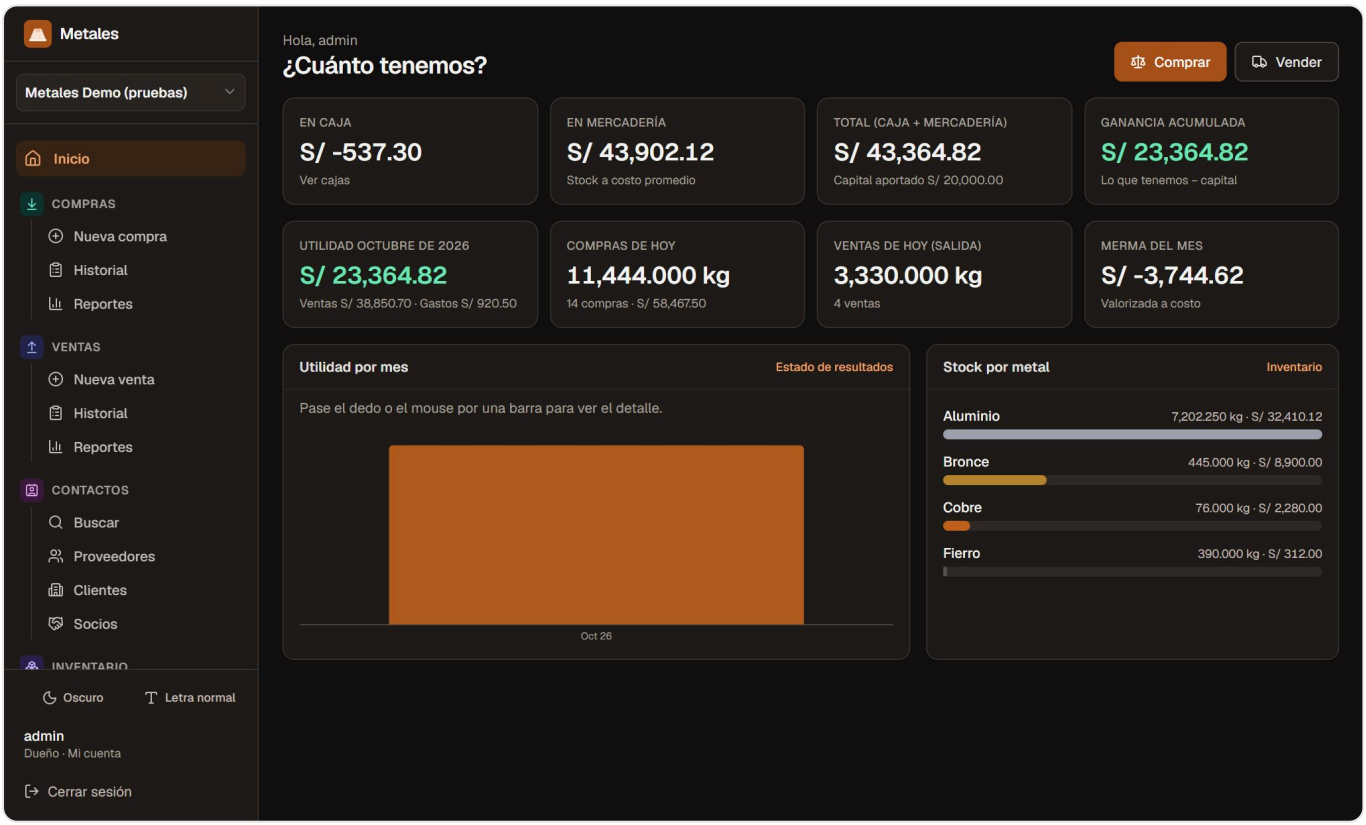
Gestión Reportes y configuración



Todos los reportes, exportables a Excel y PDF, con filtro de fechas.



Configuración: datos de la empresa, usuarios y roles, cajas, categorías de gasto, metales y el monto desde el cual se exige DNI.



Modo oscuro. También hay «letra grande». Se cambian desde el pie del menú o en Mi cuenta → Vista.

4. Roles: quién puede hacer qué

ACCIÓN	DUEÑO	SOCIO	CAJERO	BALANCERO
Registrar compras	✓		✓	✓
Registrar ventas	✓		✓	
Caja (gastos, traspasos, arqueo)	✓		✓	
Crear/editar proveedores y clientes	✓		✓	
Ver todo y reportes	✓	✓		
Anular operaciones	✓			
Editar precios del día, ajustar inventario	✓			
Socios, aportes, cierre de mes, configuración	✓			

Los permisos están en dos lugares, a propósito

1. En la pantalla (`src/lib/permisos.ts`): solo sirve para *ocultar* botones que el usuario no puede usar. Es comodidad.
2. En la base de datos (función `exigir_rol()` y las políticas RLS): esta es la **garantía real**. Aunque alguien manipule la página, la base de datos rechaza la operación. Regla: *nunca confiar solo en el frontend*.

5. Conceptos del negocio, con números

Peso bruto, descuento y peso neto

Llegan 5.250 kg de cobre, pero tiene tierra: se descuentan 0.250 kg. **Neto = 5.000 kg**. A S/ 30 el kilo, se pagan **S/ 150.00**. (Este es exactamente el caso de prueba nº 1.)

Costo promedio ponderado

El metal se compra a distintos precios en el día. ¿Cuánto nos «costó» cada kilo que tenemos? Se promedia **pesando por la cantidad**:

COMPRA	KG	PRECIO	COSTO
Bronce 1	10	S/ 20	S/ 200
Bronce 2	30	S/ 24	S/ 720
Stock	**40**	**S/ 23** (= 920 ÷ 40)	**S/ 920**

Cada compra recalcula el promedio. Cada salida (venta, merma) se valora a ese costo. Así se sabe la ganancia real de cada venta: precio de venta − costo promedio.

Merma

Vendemos 4.000 kg de bronce (costo 23). El comprador dice que recibió 3.950 kg y paga S/ 26 el kilo.

- Merma = 4.000 − 3.950 = **0.050 kg**, valorizada a costo = 0.05 × 23 = **S/ 1.15** (pérdida).
- Cobramos 3.950 × 26 = **S/ 102.70**, que entran a caja.

También hay merma por **conteo físico**: si en el inventario el sistema dice 100 kg y en la balanza hay 98, se registra un ajuste de −2 kg.

Kardex

El libro de cada metal: toda entrada y salida, con su costo y el saldo después de cada movimiento. Es lo que usan los contadores para auditar inventario.

Caja, traspaso y arqueo

- **Caja**: cada «bolsillo» de dinero (caja chica en efectivo, cuenta del banco, Yape). Cada compra, venta, gasto o aporte genera su movimiento **automáticamente**.
- **Traspaso**: mover dinero entre cajas (por ejemplo, depositar efectivo al banco). No cambia el total, solo dónde está.
- **Arqueo**: el cajero cuenta los billetes y lo anota; el sistema muestra la diferencia con lo que «debería» haber.

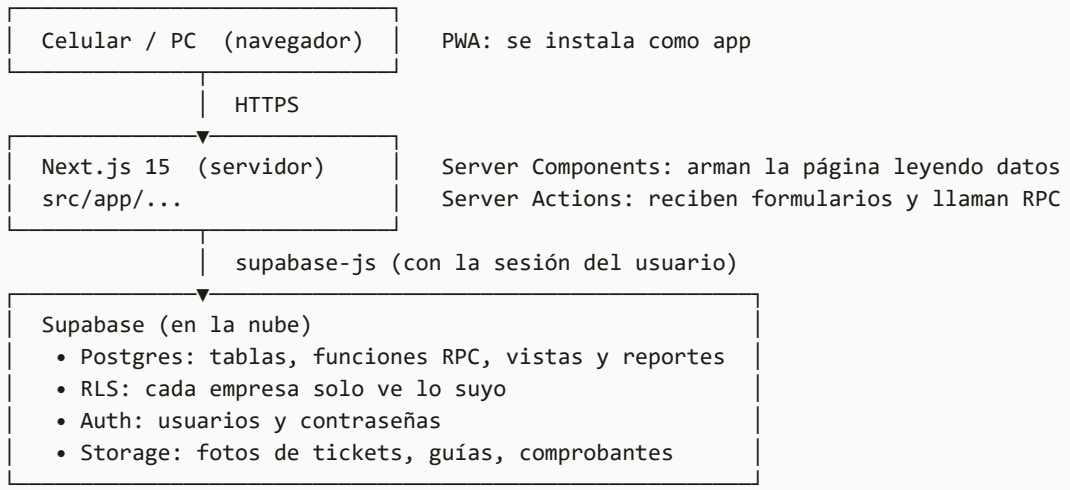
Capital, patrimonio y ganancia

Capital = lo que los socios pusieron (aportes − retiros).
Patrimonio («¿cuánto tenemos?») = caja + stock a costo + mercadería en tránsito (ventas aún no confirmadas).
Ganancia acumulada = patrimonio − capital.
Utilidad del mes = ventas − costo de lo vendido − mermas − gastos. Se reparte entre socios según su %, solo como información; el reparto real se registra como retiro.

Anular y cerrar mes

Nada se borra. Si una compra estuvo mal, se **anula con motivo**: el dinero vuelve a caja, el metal sale del stock y queda registrado quién y por qué. El **cierre de mes** toma una «foto» de los resultados y **bloquea** cualquier cambio en ese mes; solo el dueño puede reabrirlo, con motivo.

6. Arquitectura: las piezas



TECNOLOGÍA	PARA QUÉ SE USA AQUÍ
Next.js 15 (App Router)	El framework de la aplicación. Cada carpeta en `src/app` es una ruta (URL). Corre código en el servidor y en el navegador.
React 19	La librería para construir la interfaz con componentes (piezas reutilizables: botón, tarjeta, tabla).
TypeScript	JavaScript con tipos: el editor avisa errores antes de ejecutar (por ejemplo, pasar texto donde va un número).
Tailwind CSS 4	Estilos escritos como clases (`rounded-xl p-4 text-sm`). Los colores y el modo oscuro se definen en `globals.css` .
Supabase	«Backend como servicio»: base de datos Postgres + usuarios + archivos + API automática. *Nota: es Postgres, no SQLite.*
PostgreSQL	La base de datos. Aquí vive **toda la lógica de dinero e inventario**.
Zod	Valida los datos de los formularios.
@react-pdf/renderer	Genera el ticket de 80 mm y los reportes en PDF.
ExcelJS	Genera los reportes en Excel.
Vitest + PGlite	Pruebas automáticas con un Postgres que corre dentro de Node, sin internet.
pnpm	Gestor de paquetes (como npm, pero más rápido y ahorra disco).

La regla de oro del proyecto

El dinero y el inventario se calculan en la base de datos, nunca en la pantalla. La pantalla solo muestra y envía datos. ¿Por qué? Porque si el cálculo estuviera en el navegador, dos celulares a la vez, una conexión que se corta o un usuario malicioso podrían dejar la caja y el stock descuadrados. En Postgres todo ocurre en una **transacción**: o se guarda todo, o nada. (La única excepción es `src/lib/montos.ts`, que calcula la *vista previa* del total mientras se escribe; el total oficial lo calcula la BD.)

7. El viaje de una compra

Seguir una compra de principio a fin es la mejor forma de entender cómo se conectan todas las piezas.

1. **Pantalla** — `src/app/(app)/comprar/form-compra.tsx`. Es un componente de cliente (`"use client"`) porque reacciona a cada tecla: suma el total en vivo con `montos.ts` (en gramos y céntimos enteros para no tener errores de decimales).
2. **Clave única** — el formulario genera una `clave` (UUID) al abrirse. Si el celular pierde señal y reenvía, la BD reconoce la clave y devuelve la misma compra en lugar de duplicarla (*idempotencia*).
3. **Server Action** — al tocar «Pagar y guardar» se llama a `registrarCompra()` en `src/app/(app)/compras/acciones.ts`. Corre en el servidor, con la sesión del usuario.

```
// src/app/(app)/compras/acciones.ts (resumido)
"use server";
export async function registrarCompra(d: DatosCompra) {
  const { supabase, empresa } = await obtenerContexto();   // usuario + empresa activa
  const { data, error } = await supabase.rpc("registrar_compra", {
    p_empresa: empresa.id, p_items: d.items, p_caja: d.caja_id,
    p_medio_pago: d.medio_pago, p_proveedor: d.proveedor_id, p_clave: d.clave, ...
  });
  if (error) return fallo(error);       // mensaje entendible para el usuario
  revalidatePath("/", "layout");       // refresca las pantallas que muestran totales
  return ok(data);                      // { id, numero, total }
}
```

4. **Función en Postgres** — `registrar_compra()` en `supabase/migrations/…_funciones.sql`. Es `SECURITY DEFINER`: puede escribir en tablas que el usuario no puede tocar directamente, pero **primero valida el rol**.

```
-- supabase/migrations/20261016000004_funciones.sql (resumido)
create or replace function registrar_compra(p_empresa uuid, p_items jsonb, p_caja uuid, ...)
returns jsonb language plpgsql security definer as $$
begin
  v_rol := exigir_rol(p_empresa, '{dueno,cajero,balancero}');  -- ¿puede comprar en ESTA empresa?
  -- ¿ya existe una compra con esta clave? → devolverla (no duplicar)
  -- validar: al menos un metal, pesos positivos, DNI si supera el monto configurado
  -- por cada línea: peso neto, importe, insertar compra_items, _stock_entrada() → kardex + costo
promedio
  -- _mov_caja(): egreso de caja por el total (falla si la caja no es de la empresa)
  return jsonb_build_object('id', ..., 'numero', ..., 'total', ...);
end $$;
```

5. **Todo o nada** — compra, líneas, stock, kardex y movimiento de caja se guardan en **una sola transacción**. Si algo falla (por ejemplo, mes cerrado), no queda nada a medias.
6. **Respuesta** — vuelve `{ id, numero, total }`; la pantalla muestra el número de compra y el botón del ticket PDF (`src/lib/pdf/ticket.tsx`).
7. **Lectura** — el inicio, el historial y los reportes leen **vistas** (`v_compras`, `v_stock_metal`, `v_patrimonio`…) desde Server Components. Las vistas ya traen los números calculados.

El mismo patrón se repite en todo el sistema

Pantalla → Server Action → función RPC en Postgres → vistas para leer. Vender, anular, gastos, traspasos, aportes y cierres siguen exactamente este camino. Si entiendes la compra, entiendes el sistema.

8. La base de datos por dentro

El esquema está en `supabase/migrations/`, en 7 archivos que se aplican en orden. Una **migración** es un archivo SQL que lleva la base de un estado al siguiente. Regla: **una migración ya subida no se edita**; si hay que cambiar algo, se crea una nueva.

MIGRACIÓN	QUÉ CONTIENE
`…0001_base.sql`	Tipos (roles, estados, medios de pago) y funciones de apoyo.
`…0002_tablas.sql`	Las 22 tablas.
`…0003_rls.sql`	Seguridad: políticas RLS por empresa y trigger que impide borrar.
`…0004_funciones.sql`	Las operaciones: registrar/anular compra, venta, gasto, traspaso, aporte, cierre…
`…0005_vistas_reportes.sql`	Vistas `v_*` y reportes `rpt_*`.
`…0006_storage_admin.sql`	Archivos privados por empresa y alta de empresas y miembros.
`…0007_resumenes_modulo.sql`	Resúmenes de cada módulo y fichas «por metal».

Tablas (agrupadas)

GRUPO	TABLAS
Empresa y usuarios	`empresas`, `miembros` (usuario + empresa + rol), `correlativos` (numeración)
Catálogos	`metales`, `precios_metal` (histórico), `cajas`, `categorias_gasto`
Contactos	`proveedores`, `compradores`, `socios`
Operación	`compras` + `compra_items`, `ventas` + `venta_items`, `ajustes_inventario`
Inventario	`stock_metal` (kg y costo actual), `kardex` (libro)
Dinero	`movimientos_caja`, `gastos`, `traspasos`, `arqueos`, `aportes_retiros`, `cierres_mes`

Funciones (las «operaciones»)

Públicas (las llama la app): `registrar_compra`, `anular_compra`, `registrar_venta`, `confirmar_venta`, `anular_venta`, `registrar_gasto`, `registrar_traspaso`, `registrar_arqueo`, `registrar_ajuste`, `registrar_conteo`, `reclasificar_metal`, `registrar_aporte_retiro`, `cerrar_mes`, `reabrir_mes`, `dashboard`, `resumen_compras/ventas/inventario`, `proveedor_por_metal`, `comprador_por_metal`.

Internas (empiezan con `_`, no se exponen): `_stock_entrada`, `_stock_salida` (valoriza al céntimo), `_mov_caja`, `_validar_caja`, `_fecha_operacion`.

Seguridad: `exigir_rol`, `tiene_rol`, `es_miembro`, `mi_rol`. **Triggers**: `trg_no_borrar` (impide DELETE), `trg_periodo_abierto` (bloquea meses cerrados), `trg_metal_precio_historico`.

Vistas y reportes (para leer)

`v_compras`, `v_ventas`, `v_stock_metal`, `v_kardex`, `v_saldo_caja`, `v_movimientos_caja`, `v_resultado_mes`, `v_patrimonio`, `v_capital_socio`, `v_utilidad_socio`, `v_proveedores_resumen`, `v_compradores_resumen` … y los reportes `rpt_compras`, `rpt_ventas`, `rpt_margen_metal`, `rpt_merma`, `rpt_libro_caja`, `rpt_ranking_proveedores`.

Tipos de dato elegidos

Montos en `numeric(14,2)` (soles con 2 decimales exactos) y pesos en `numeric(12,3)` (gramos). **Nunca** se usan números de punto flotante para dinero: `0.1 + 0.2` en JavaScript da `0.30000000000000004`.

9. Seguridad

MEDIDA	QUÉ PROTEGE
RLS (Row Level Security)	Postgres filtra cada consulta: un usuario solo ve filas de empresas donde es miembro. Aunque el código tuviera un error, la base no entrega datos ajenos. Probado en los tests.
Sin escritura directa	Las tablas de operación no tienen políticas de escritura: solo se modifican a través de las funciones, que validan el rol con `exigir_rol()`.
Nada se borra	Un trigger impide `DELETE`. Se anula con motivo y usuario: queda auditoría.
Mes cerrado	Un trigger bloquea registrar o anular en meses cerrados.
Claves secretas	Viven solo en `.env.local` (ignorado por Git). La `service_role key` solo se usa en el servidor, nunca llega al navegador. `.env.example` documenta qué variables hacen falta, sin valores.
Tokens fuera del chat	El token de Supabase y la contraseña de la BD se escribieron con scripts ocultos (`pnpm supabase:conectar`, `pnpm supabase:clave`), nunca pegados en una conversación.
Acceso directo solo en desarrollo	El botón «Entrar directo» del login solo existe con `NODE_ENV=development`; en producción la ruta responde 404.
Archivos privados	Fotos y guías en un bucket privado, en carpetas por empresa (`archivos/{empresa_id}/…`).

Pendiente recomendado

En Supabase → Authentication, desactivar «Allow new users to sign up» para que nadie pueda crearse una cuenta por su cuenta; los usuarios los crea el dueño desde Configuración.

10. Estructura de carpetas

```
Proyecto_Otiniano/
├─ src/
│  ├─ app/                          ← cada carpeta es una URL
│  │  ├─ layout.tsx                 ← HTML base, tema claro/oscuro
│  │  ├─ globals.css                ← colores, modo oscuro, letra grande
│  │  ├─ login/                     ← /login
│  │  ├─ auth/                      ← salir, acceso de desarrollo
│  │  ├─ api/consulta-doc/          ← consulta DNI/RUC (falta el token de la API)
│  │  └─ (app)/                     ← todo lo que requiere sesión (el paréntesis no sale en la URL)
│  │     ├─ layout.tsx              ← verifica sesión y arma el menú
│  │     ├─ page.tsx                ← / inicio (dueño) · inicio-operativo.tsx (cajero/balancero)
│  │     ├─ comprar/  compras/      ← nueva compra · historial, detalle [id], reportes
│  │     ├─ vender/   ventas/
│  │     ├─ contactos/ proveedores/ compradores/ socios/
│  │     ├─ inventario/ precios/
│  │     ├─ caja/  capital/
│  │     ├─ reportes/  configuracion/  cuenta/
│  ├─ components/
│  │  ├─ ui/                        ← botones, campos, tabla→tarjetas, modal, pestañas
│  │  ├─ shell/                     ← modulos.ts + menú lateral + barra inferior
│  │  ├─ reportes/  preferencias/  resumen-modulo.tsx  graficos.tsx
│  └─ lib/
│     ├─ contexto.ts                ← quién es el usuario, su empresa y su rol
│     ├─ permisos.ts                ← qué botones mostrar según el rol
│     ├─ acciones.ts                ← ok()/fallo(): respuesta estándar de las acciones
│     ├─ formato.ts  montos.ts  reportes.ts  preferencias.ts  dev-login.ts
│     ├─ pdf/                       ← ticket 80 mm y reporte A4
│     └─ supabase/                  ← clientes de Supabase y tipos generados
├─ supabase/
│  ├─ migrations/                   ← el esquema (7 archivos)
│  └─ scripts/                      ← 01_crear_empresa.sql, 02_datos_demo.sql
├─ scripts/                         ← conectar Supabase, cambiar clave, validar migraciones
├─ tests/                           ← pruebas automáticas
├─ docs/                            ← capturas y esta documentación
├─ CLAUDE.md                        ← reglas del proyecto para el asistente
├─ PROPUESTA_SISTEMA_CHATARRA.md    ← la especificación original
└─ .env.local                       ← secretos (NO se sube a GitHub)
```

Server Components vs. Client Components

En Next.js, por defecto los componentes corren **en el servidor**: leen la base de datos y mandan HTML ya armado (ejemplo: el historial de compras). Solo los que necesitan interactividad (escribir, tocar, abrir un modal) llevan `"use client"` al inicio y corren en el navegador (ejemplo: el formulario de compra). Ventaja: menos código en el celular y las claves nunca salen del servidor.

11. Pruebas automáticas

`pnpm test` levanta un Postgres **dentro de Node** (PGlite), aplica las 7 migraciones y ejecuta los casos de negocio de la propuesta. No necesita internet ni tocar la base real.

PRUEBA	QUÉ ASEGURA
Empresa B no ve nada de la A	El aislamiento multiempresa (RLS) funciona.
B no puede comprar en A	Las funciones validan la empresa.
No se escribe directo en compras ni en caja	Solo se escribe vía funciones.
El cajero no edita precios ni anula	Los roles se respetan en la BD.
1. Compra 5.250 kg − 0.250 a S/ 30	Total S/ 150.00, caja −150, stock +5 kg.
2. Dos compras a distinto precio	Costo promedio ponderado = 23.
3. Venta con merma	Merma 0.050 kg = S/ 1.15; cobro S/ 102.70.
4. Vender más que el stock	La BD lo rechaza con un mensaje claro.
5. Anular una compra	Caja y stock vuelven exactamente; queda quién y por qué.
6. Cierre de mes	No se puede registrar ni anular sin reabrir.
Patrimonio	Caja + stock + tránsito − capital = suma de utilidades, al céntimo.
Datos demo	Los scripts de alta y demo cargan y todo cuadra, incluidos los resúmenes de módulo.
PDF (4 pruebas)	Ticket de 80 mm, reporte A4 y redondeo igual al de Postgres.

Antes de dar algo por terminado se corre siempre: `pnpm typecheck` (tipos), `pnpm lint` (estilo y errores comunes), `pnpm test` y `pnpm build` (que compile para producción).

12. Qué se hizo, en orden

1. **Análisis de la propuesta.** Se leyó `PROPUESTA_SISTEMA_CHATARRA.md`, se resumió y se armó un plan por sprints (0 a 7). Se aclaró que Supabase usa Postgres, no SQLite, y que el producto es un sistema privado (PWA), sin web pública.
2. **Construcción del MVP completo.** Proyecto Next.js con pnpm, las 6 primeras migraciones (tablas, RLS, funciones, vistas, storage), todas las pantallas, PDF, Excel, roles y pruebas.
3. **Conexión con Supabase en la nube.** Scripts para escribir el token y la contraseña de forma oculta, enlazar el proyecto y subir las migraciones. Creación de la empresa real y del usuario dueño.
4. **Servidor local y acceso directo de desarrollo.** Botón «Entrar directo» en el login, solo en desarrollo.
5. **Datos de prueba.** Empresa separada «Metales Demo (pruebas)» con proveedores, compras, ventas y movimientos inventados, para no ensuciar la empresa real.
6. **Reorganización visual por módulos.** Compras, Ventas, Inventario, Caja y finanzas, cada uno con sus pestañas (Nueva · Historial · Reportes) y reportes propios.
7. **Cinco mejoras de diseño.** Inicio según el rol, resúmenes de cada módulo calculados en la BD (migración 7), colores por módulo, fichas de proveedor y cliente con pestañas y «por metal», modo oscuro y letra grande.
8. **Módulo Contactos.** Buscador único por DNI, RUC o nombre; filtros de activos en proveedores, clientes y socios; ficha del socio; el dinero de los socios pasó a «Capital y cierres».
9. **Documentación.** README para GitHub (se corrigió una línea en otra codificación que lo dañaba) y este PDF.

Decisiones importantes que conviene recordar

- **Lo marcado [CONFIRMAR] no se inventó**: se dejó configurable (monto desde el que se pide DNI, quién cambia precios, metales, cajas, socios).
- **Los datos de prueba están en otra empresa**, para que la real empiece limpia.
- **No se copió nada del ERP de transportes** (ni datos, ni .env, ni tokens).

13. Cómo se trabaja en el día a día

Comandos

COMANDO	PARA QUÉ
`pnpm install`	Instala dependencias (la primera vez o cuando cambie `package.json`).
`pnpm dev`	Levanta el sistema en `http://localhost:3000`. Se actualiza solo al guardar un archivo.
`pnpm test`	Corre las pruebas. Obligatorio después de cualquier cambio en SQL.
`pnpm typecheck` · `pnpm lint`	Revisan errores de tipos y de estilo.
`pnpm build`	Compila para producción. Si falla aquí, fallaría en Vercel. (No correrlo con `pnpm dev` encendido.)
`pnpm db:push`	Sube migraciones nuevas a Supabase.
`pnpm db:types`	Regenera los tipos TypeScript desde la base.

Para agregar una funcionalidad

1. **¿Toca dinero o inventario?** Crear una migración nueva en `supabase/migrations/` con la función o vista.
2. Agregar su prueba en `tests/db/` y correr `pnpm test`.
3. Crear la pantalla: un Server Component que lee la vista y una Server Action que llama la función.
4. Si es una pantalla nueva, agregarla a `src/components/shell/modulos.ts`.
5. Verificar: `pnpm typecheck && pnpm lint && pnpm test && pnpm build`.
6. Probar en el celular; luego `pnpm db:push` y desplegar.

Git y GitHub

El código está en `github.com/bravoacamus-droid/prueba-otoniano`. Ciclo básico:

```
git status                    # ¿qué cambió?
git add .                     # preparar todo (los secretos están en .gitignore)
git commit -m "Describe el cambio"
git push                      # subir a GitHub
```

Ojo: antes de `git add .`, mirar `git status` y confirmar que no aparecen `.env.local` ni `credenciales-dueno.txt`. Ambos ya están ignorados.

Sobre el README: si se edita en Windows, evitar `echo "texto" >> README.md` en PowerShell 5: escribe en UTF-16 y GitHub lo muestra con letras separadas. Editarlo siempre desde VS Code.

14. Pendientes y decisiones abiertas

Siguiente paso inmediato

- **API de SUNAT/RENIEC:** al escribir un DNI o RUC en «Nuevo proveedor» o «Nuevo cliente», traer automáticamente nombre o razón social y dirección. Ya existe la ruta `src/app/api/consulta-doc`; falta el proveedor de API y su token (que se guarda en `.env.local`, nunca en el chat).

Preguntas al dueño

- Al confirmar una venta, ¿se debe **bloquear o solo avisar** si el peso recibido es mayor al de salida? (En las pruebas se registró 5,000 kg recibidos contra 300 kg enviados.)
- ¿Se permite **pagar sin saldo** en caja? (Hoy la caja chica de prueba quedó negativa.)
- Del documento original **[CONFIRMAR]**: ventas a crédito, ventas en dólares, varios locales, compras por unidad (baterías, motores), monto desde el que se pide DNI.

Antes de usarlo en el local

- Desactivar el registro libre de usuarios en Supabase.
- Probar en un celular real (instalar como app, imprimir el ticket, foto de balanza).
- Desplegar en Vercel y configurar la URL en Supabase.
- Decidir si se desactivan los usuarios de prueba (cajero.demo y balancero.demo).

Fase 4 (futuro)

- Facturación y guías electrónicas SUNAT.
- Sorteos y premios para proveedores frecuentes (sobre el ranking ya existente).
- Versión SaaS: que otros negocios se registren solos (la base multiempresa ya está lista).

15. Ruta de estudio y glosario

Para dominar este proyecto, conviene estudiar en este orden. Cada tema tiene un archivo del proyecto donde verlo aplicado:

#	TEMA	DÓNDE VERLO EN EL PROYECTO
1	SQL básico (select, insert, join, group by)	`supabase/migrations/…_vistas_reportes.sql`
2	Funciones PL/pgSQL y transacciones	`registrar_compra` en `…_funciones.sql`
3	RLS y roles en Postgres/Supabase	`…_rls.sql` y `exigir_rol()`
4	TypeScript básico (tipos, interfaces)	`src/lib/tipos.ts` , `src/lib/permisos.ts`
5	React: componentes, props, estado (`useState`)	`src/components/ui/` , `comprar/form-compra.tsx`
6	Next.js App Router: rutas, layouts, Server Components	`src/app/(app)/layout.tsx` , `compras/page.tsx`
7	Server Actions y formularios	`compras/acciones.ts` , `src/lib/acciones.ts`
8	Tailwind CSS y variables de color	`src/app/globals.css` , `components/ui/clases.ts`
9	Pruebas con Vitest	`tests/db/casos.test.ts`
10	Git y GitHub	Este repositorio

Glosario técnico

TÉRMINO	SIGNIFICADO
Frontend / Backend	Lo que ve el usuario (pantallas) / lo que procesa y guarda datos (servidor y base).
PWA	Progressive Web App: una web que se instala como app en el celular.
RPC	«Llamada a procedimiento remoto»: desde la app se ejecuta una función que vive en la base de datos.
Vista (view)	Una consulta guardada con nombre; se lee como si fuera una tabla.
Migración	Archivo SQL versionado que cambia la estructura de la base.
RLS	Row Level Security: reglas que filtran qué filas puede ver o tocar cada usuario.
SECURITY DEFINER	Función que corre con permisos de su dueño (no del usuario); por eso valida el rol primero.
Transacción	Grupo de cambios que se guardan todos juntos o ninguno.
Idempotencia	Repetir la misma operación no cambia el resultado (no duplica la compra).
Trigger	Código que la base ejecuta sola ante un evento (por ejemplo, impedir un DELETE).
Server Component	Componente React que corre en el servidor y entrega HTML listo.
Server Action	Función del servidor que se llama desde un formulario, sin crear una API a mano.
Variables de entorno	Configuración y secretos fuera del código (`.env.local`).
Token / API key	Contraseña para que un programa use un servicio. Nunca se comparte ni se sube a GitHub.
Deploy	Publicar la aplicación en un servidor (Vercel) para usarla desde internet.

Consejo final

Abre el proyecto en VS Code, levanta `pnpm dev` y recorre una compra con este documento al lado: pantalla → `acciones.ts` → `registrar_compra` → vista. Después cambia algo pequeño (un texto, un color) y mira cómo se actualiza. Así se aprende más rápido que leyendo.